

Degree Project in Computer Science and Engineering, specialising in Embedded Systems

Second cycle, 30 credits

AI-Based Frequency Selector with Input Metrics Exploration for DVFS in NoC for Many-Core Systems

ZIYIN YANG

AI-Based Frequency Selector with Input Metrics Exploration for DVFS in NoC for Many-Core Systems

ZIYIN YANG

Master's Programme, Embedded Systems, 120 credits
Date: June 23, 2026

Supervisor: Yizhi Chen
Examiner: Ahmed Hemani

Swedish title: AI-Baserad Frekvensväljare med Utforskning av Indatametrik för DVFS i NoC för Mångkärniga System

© 2026 Ziyin Yang

Abstract

Network-on-Chip (NoC) interconnects carry most of the on-chip communication in many-core processors, and their efficiency affects both throughput and energy use. Because traffic varies across workloads and over the course of a run, a fixed router frequency rarely matches the actual load, which motivates adaptive Dynamic Voltage and Frequency Scaling (DVFS) at the router level.

Most existing adaptive NoC DVFS controllers are hand-written rule sets based on empirical observation. Their fixed thresholds are tuned for specific conditions and transfer poorly to other workloads, motivating a controller learned directly from simulation data.

This thesis replaces the rule set with a Convolutional Neural Network (CNN) that predicts the router frequency from a short history of network-state observations. Training labels come from full-system gem5/Garnet simulations of PARSEC benchmarks: at each decision point, each candidate frequency is scored by a reward function that weights throughput gain relative to a 1.0 GHz baseline against a quadratic power cost and rejects any choice that violates a latency bound. Three CNNs are trained with different power-performance trade-off weights, giving controllers that range from performance-oriented to power-conservative.

In closed-loop simulation on the canneal benchmark, the three CNNs produce different control behavior, confirming that the trade-off weight yields distinct policies. The HighPerf-CNN variant runs fastest, completing canneal in 86.5% of the 1.0 GHz baseline time and slightly beating the hand-tuned rule-based controller (87.4%) without manual threshold tuning. A learned controller can therefore match or marginally exceed a hand-engineered one, and the preferred variant depends on whether throughput or more conservative frequency selection is prioritized.

Keywords

Network-on-Chip, Dynamic Voltage and Frequency Scaling, Convolutional Neural Network, Full-system simulation, Adaptive frequency control

Sammanfattning

Nätverk-på-chip-förbindelser (NoC) hanterar merparten av kommunikationen på chippet i flerkärniga processorer, och deras effektivitet påverkar både genomströmning och energiförbrukning. Eftersom trafiken varierar mellan olika arbetsbelastningar och under en körnings gång, matchar en fast routerfrekvens sällan den faktiska belastningen. Detta motiverar adaptiv dynamisk spännings- och frekvensskalning (DVFS) på routernivå.

De flesta befintliga adaptiva NoC DVFS-styrenheter är handskrivna regelverk baserade på empiriska observationer. Deras fasta tröskelvärden är anpassade för specifika förhållanden och fungerar dåligt för andra arbetsbelastningar, vilket motiverar en styrenhet som lärs upp direkt från simuleringsdata.

Denna avhandling ersätter regelverket med ett faltningsnätverk (Convolutional Neural Network, CNN) som förutsäger routerfrekvensen utifrån en kort historik av nätverkstillståndsobservationer. Träningsetiketterna hämtas från fullsystemssimuleringar med gem5/Garnet av PARSEC-riktmärken: vid varje beslutspunkt poängsätts varje frekvenskandidat av en belöningsfunktion. Denna funktion väger genomströmningsökningen – jämfört med en baslinje på 1,0 GHz – mot en kvadratisk effektkostnad, och förkastar alla val som överskrider en fastställd latensgräns. Tre CNN:er tränas med olika vikter för avvägningen mellan effekt och prestanda, vilket resulterar i styrenheter som sträcker sig från prestandaorienterade till effektkonservativa.

I sluten simulering på canneal-riktmärket producerar de tre CNN:erna olika kontrollbeteenden, vilket bekräftar att avvägningsvikten ger distinkta policyer. Den prestandaorienterade varianten körs snabbast, slutför canneal på 86,5% av baslinjen på 1,0 GHz och slår den handjusterade regelbaserade regulatorn (87,4 %) något utan manuell tröskeljustering. En inlärd regulator kan därför matcha eller marginellt överträffa en handkonstruerad, och den föredragna varianten beror på om genomströmning eller mer konservativt frekvensval prioriteras.

Nyckelord

Nätverk på chip, Dynamisk spännings- och frekvensskalning, Faltningsneuralt nätverk, Fullsystemsimulering, Adaptiv frekvenskontroll

Acknowledgments

Writing these words on a bright night, Stockholm has become a city without sunset again, like the first day we met. These two years have been a challenging but rewarding journey, shaping me both academically and personally.

I would like to thank my supervisor, Yizhi Chen, for the continued instruction. This project would not have been possible without his direction and technical discussions. I really learned a lot from Yizhi beyond the technical direction, both professional knowledge and the way to approach research problems systematically, which I will carry forward in my future work.

I would like to thank my best neighbors and friends, Huakun Chen, Yue Li, and Dayu Zhang. From hotpot dinners, long conversations, skating on frozen lakes and sledding down snowy hills, the time we shared in Stockholm has given me memories I will carry long after leaving.

I am also deeply grateful to my friends Xiaotong Li, Ruiling Xu, Ruichi Han, Tong Lei, Yurun Chao, Xina Luo, as well as everyone who has touched this journey in ways big or small. The memories we created together have made me certain that choosing to come here was the right decision.

I would also like to thank my family and old friends for their support and encouragement throughout this journey. Your support makes me keep going, and I look forward to reuniting back home.

During the writing of this thesis, AI was used to assist with polishing sentences. For figures, AI was only used to generate Python plotting code; all contents, such as data, labels, and units are my own work. AI was also used to assist with generating initial code drafts and debugging. All design decisions and functional requirements were determined by me, and the generated code was reviewed and modified as needed. All technical content, results, analysis, and conclusions are my own work. No AI-generated text has been included without review and revision.

Stockholm, June 2026
Ziyin Yang

Contents

1 Introduction **1**
1.1 Background . . . 1
1.2 Problem . . . 2
1.2.1 Original problem and definition . . . 2
1.2.2 Scientific and engineering issues . . . 3
1.3 Purpose . . . 3
1.4 Goals . . . 4
1.5 Research Methodology . . . 5
1.6 Structure of the thesis . . . 5

2 Background **7**
2.1 Network-on-Chip . . . 7
2.1.1 Motivation and Architecture . . . 7
2.1.2 Performance Metrics . . . 8
2.2 Dynamic Voltage and Frequency Scaling . . . 9
2.3 Multi-core simulation framework . . . 10
2.3.1 gem5 Simulator . . . 10
2.3.2 Garnet NoC Model . . . 11
2.3.3 Full-System Simulation and PARSEC Benchmarks . . . 12
2.4 ML in DVFS . . . 14
2.4.1 One-Dimensional Convolutional Neural Network . . . 14
2.4.2 Model Export and Runtime Inference . . . 14
2.5 Related work area . . . 16
2.5.1 Rule-Based Flow Control . . . 16
2.5.2 AI-Based NoC Control . . . 17

3 Methods **19**
3.1 Research Process and Research Paradigm . . . 19
3.2 Simulation Platform . . . 20

3.2.1 Test Environment 20
3.2.2 NoC Simulation Configuration 20
3.3 Data Collection 20
3.3.1 Synthetic Traffic Simulation 20
3.3.2 Full-System Simulation 21
3.4 Rule-Based Controller 22
3.4.1 Choice of Input Features 22
3.4.2 Indicator Design 23
3.5 CNN-Based DVFS Controller 23
3.5.1 Label Generation 23
3.5.2 Feature Engineering and Normalisation 24
3.5.3 Model Architecture and Training 26
3.5.4 Ablation Study Design 27
3.5.4.1 Feature Group Ablation 28
3.5.4.2 History Window Ablation 28
3.5.5 Deployment in gem5 29
3.6 Evaluation Framework 29

4 System Implementation **31**
4.1 Overall System Architecture 31
4.2 Runtime Integration in gem5 32
4.2.1 Clock Domain and Frequency Scaling 32
4.2.2 Network Observation: `FlowObserver` 34
4.2.3 Frequency Decision: `FlowRegulator` 34
4.2.4 Training Data Collection 35
4.2.5 Rule-Based Controller Implementation 35
4.2.5.1 Threshold Values 36
4.2.5.2 Algorithmic Structure 37
4.3 Offline Training Pipeline 37
4.3.1 Label Generation and λ Sweep 37
4.3.2 Model Selection 39
4.3.3 Ablation Study 41
4.3.3.1 Feature Group Ablation 41
4.3.3.2 History Window Ablation 41
4.4 Evaluation Setup 42

5 Results and Analysis **43**
5.1 CNN Controller Training Results 43
5.1.1 Cross-Validation Performance 43

5.1.2 Training Dynamics 44
5.1.3 Confusion Matrix Analysis 46
5.1.4 Ablation Study 47
5.1.4.1 Feature Group Ablation 47
5.1.4.2 History Window Ablation 48
5.2 NoC DVFS Evaluation 49
5.2.1 NoC Performance Characterisation 49
5.2.2 Fixed-Frequency Benchmark Evaluation 53
5.2.3 Controller Performance 54
5.3 Summary 55

6 Conclusions and Future Work **57**
6.1 Conclusions 57
6.2 Limitations 58
6.3 Future Work 59

References **61**

A Checkpoint Workflow Details **67**
A.1 Checkpoints Workflow 67

List of Figures

1.1 Growth in maximum single-socket core count for x86 server processors, 2016–2024 [4, 5, 6, 7, 2, 8, 9, 10, 11, 12]. 2

2.1 Examples of Ring, Star, Mesh, and Torus network structures, reproduced from [21]. 8
2.2 Architecture of the gem5 simulator 11

3.1 Research workflow: controller design and evaluation pipeline . 19
3.2 Three-phase checkpoint strategy for full-system gem5 simulation. 22
3.3 Architecture of the CNN controller. 27

4.1 Overall architecture of the CNN-based DVFS control system. . 32
4.2 Logical Wakeup Spacing (1.5 GHz on 10 GHz Base). 33
4.3 Trajectory structure for training data collection. 35
4.4 Effect of λ on label distribution and cluster agreement. 38
4.5 K-means analysis ($K = 3, \lambda = 0.10, N = 1117$) on standardised features via PCA. (a) Reward-based labels; (b) K-means clusters remapped to frequency labels; (c) Confusion matrix (ARI = 0.286, agreement = 66.3 %); (d) Spatial agreement map. 40

5.1 Training loss and validation balanced accuracy curves for the HighPerf-CNN model. 44
5.2 Training loss and validation balanced accuracy curves for the Balanced-CNN model. 45
5.3 Training loss and validation balanced accuracy curves for the LowPower-CNN model. 45
5.4 Confusion matrices of each CNN controller. 46
5.5 Feature group ablation results. 48
5.6 History window ablation results ($\lambda = 0.05$, 3-fold CV). 49

5.7 Throughput and total packet latency under five fixed frequencies. 50
5.8 Network latency and queueing latency under five fixed frequencies. 51
5.9 Per-router buffer read activity in an 8×8 mesh NoC under synthetic traffic (1.0 GHz, injection rate 0.084). 52
5.10 Execution time relative to the 1.0 GHz baseline for each benchmark across fixed-frequency configurations. 53
5.11 Average packet network latency and DRAM memory access latency across fixed-frequency configurations. 54
5.12 Execution time improvement of DVFS control strategies . . . 55

List of Tables

2.1 Work units contained in PARSEC simulation input sets for each benchmark, adapted from [27]. 13
2.2 PARSEC benchmark characterization with large input sets, adapted from [28]. 13

3.1 Per-window raw feature set ($H = 2$, total input dimension $= 28$). 25
3.2 Feature groups used in the ablation study. 28

4.1 FlowObserver operating modes. 34
4.2 Threshold parameters of the rule-based baseline. 36
4.3 Selected λ values, label distributions, and cross-validation performance. Label percentages are computed over the full labelled dataset of 1,117 samples. Balanced accuracy (BalAcc) is the mean over three folds; standard deviation is shown in parentheses. 39

5.1 3-fold cross-validation results for the three CNN controllers. . 43

List of acronyms and abbreviations

1D-CNN	One-dimensional Convolutional Neural Network
BW	Buffer Write
CMOS	Complementary Metal-Oxide-Semiconductor
CMP	Chip-Multiprocessor
CPU	Central Processing Unit
DOR	Dimension-Order Routing
DVFS	Dynamic Voltage and Frequency Scaling
NI	Network Interface
NoC	Network-on-Chip
ONNX	Open Neural Network Exchange
ORT	ONNX Runtime
PARSEC	Princeton Application Repository for Shared-Memory Computers
PE	Processing Element
PVT	Process-Voltage-Temperature
RC	Route Compute
ROI	Region of Interest
SA	Switch Allocation
SLICC	Specification Language for Implementing Cache Coherence
SoC	System-on-Chip
ST	Switch Traversal
VA	Virtual Channel Allocation
VC	Virtual-Channel

Chapter 1

Introduction

1.1 Background

Demand for computing from cloud services, scientific simulation, and machine learning inference has pushed processor designs toward integrating tens to hundreds of cores on a single chip. AMD's fourth-generation EPYC Genoa processors integrate up to 96 Zen 4 cores per socket [1], and Intel's Xeon 6 Sierra Forest reaches 288 Efficient-cores per socket [2]. Adding cores, however, does not yield a proportional gain in useful throughput. As core counts rise, the cost of moving data between cores comes to dominate both performance and energy consumption [3]

This thesis focuses on the Network-on-Chip (NoC), and one mechanism for managing its power, Dynamic Voltage and Frequency Scaling (DVFS). This section provides the necessary background on both, then explains why traditional DVFS control is limited and what motivates an AI-driven alternative.

As core counts grow, the shared bus or crossbar used in small designs no longer scales. A bus has fixed bandwidth no matter how many devices attach to it, so once more than a handful of cores share it, it becomes a bottleneck [13]. The NoC replaces the bus with a packet-switched network of routers, usually arranged as a 2D mesh [14]. Data is split into fixed-size flits and forwarded from one router to the next until it reaches its destination.

DVFS is a common technique for trading performance against power, but it is harder to apply to the NoC than to a processor [15, 16]. Processor DVFS responds to coarse signals such as instructions per cycle, averaged over millisecond windows. NoC traffic changes much faster, and it depends on what all the active cores are doing at once rather than any single core. The difficulty

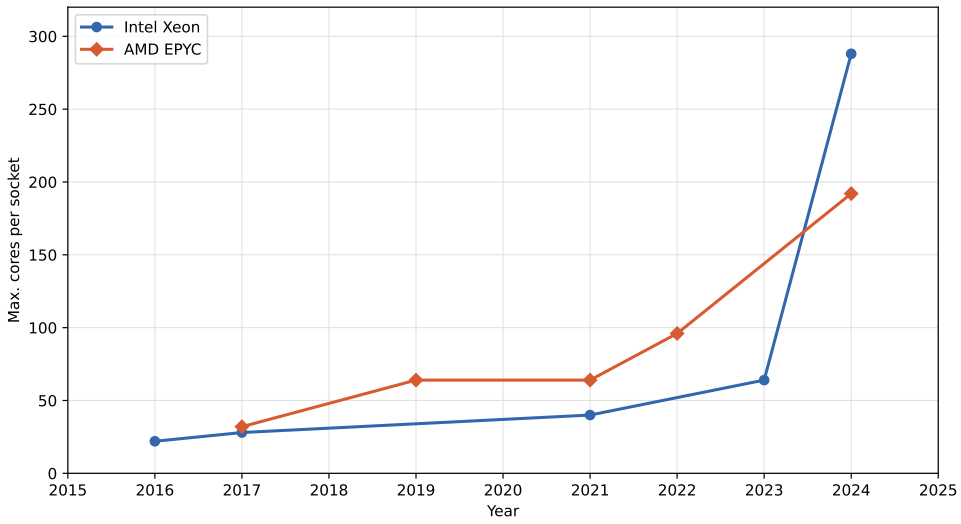


Figure 1.1: Growth in maximum single-socket core count for x86 server processors, 2016–2024 [4, 5, 6, 7, 2, 8, 9, 10, 11, 12].

is getting the timing of the frequency changes right. If the controller reacts too slowly, traffic bursts build up and latency increases. If it reacts too eagerly, it switches frequencies more often than necessary and wastes energy.

Some existing NoC DVFS controllers use hand-crafted rule bases [15]. These are cheap to run and easy to interpret, but they generally do not perform better than the rules chosen at design time. An AI-driven controller takes a different approach. Instead of fixed rules, a model is trained on examples that pair a network state with the frequency that minimizes a chosen cost, and it can learn relationships that are hard to capture in a compact rule base. The work of building such a controller, from collecting data and defining labels to training the model and integrating it into the simulator, is the subject of this thesis.

1.2 Problem

1.2.1 Original problem and definition

Traditional NoC DVFS controllers rely on static, hand-crafted rule bases that do not generalize well across the range of traffic patterns produced by real many-core workloads.

The rules are not wrong for the workloads they were tuned on; the

issue is that real workloads change a great deal over time. In compute-intensive phases, inter-core traffic is light, and a low frequency is enough. At synchronization barriers, many cores inject traffic simultaneously, and the network can quickly saturate. A fixed rule base cannot follow these changes finely enough, so it tends to run the frequency too high when traffic is light, wasting energy, or too low during bursts, which lets queueing delay build up.

This thesis asks whether a trained machine learning model can replace the hand-crafted rule base within the evaluated simulation setting and achieve a better trade-off between execution time performance and approximate power cost.

1.2.2 Scientific and engineering issues

Building an ML-based DVFS controller for the NoC raises several problems.

A first issue is how to define the frequency labels that supervised training needs. Obtaining latency-optimal labels means simulating every candidate frequency at every traffic configuration and picking the best. This sweep is expensive, and it also raises questions about how fine the labels should be and what to do when several frequency levels give almost the same latency.

There are also engineering problems. A model trained in PyTorch cannot be embedded directly in gem5's C++ codebase, so it has to be exported to ONNX, with data types matched across the language boundary and the normalization statistics built into the C++ controller.

Finally, full-system evaluation is slow. A single PARSEC run takes a long time on one server, so only a limited number of experimental conditions can be compared within the project timeline, which makes it necessary to decide carefully what to test.

1.3 Purpose

The purpose of this thesis is to design and evaluate an AI-driven DVFS controller for NoC and compare its behavior with a rule-based baseline under selected full-system many-core workloads.

The most direct beneficiaries are computer-architecture researchers working on power management for many-core systems. The pipeline built in this project, which goes from data collection through model training to C++ deployment in gem5, can serve as a reusable template for applying supervised learning in gem5-based simulation studies. The work is also relevant to processor designers and to high-performance computing more

generally, where on-chip interconnect power takes up a growing share of total chip power as core counts increase.

The project simulates a hardware subsystem and raises no significant ethical concerns. In terms of sustainability, reducing NoC power consumption fits the energy-efficiency goals of many-core design. Because all results come from simulation with a simplified power model, the claim made here is modest: the work provides evidence that motivates hardware validation rather than replacing it.

1.4 Goals

The overall goal of this degree project is to develop and evaluate an AI-driven DVFS controller for NoC implemented within the gem5 simulation framework, and to compare its latency behavior and frequency-selection tendencies against fixed-frequency and rule-based baselines. This has been divided into the following four sub-goals:

1. To reproduce a rule-based logic DVFS controller within gem5 and establish a simulation baseline against which the AI-driven approach can be systematically compared.

2. To design, train, and integrate a CNN-based DVFS controller within the same simulation environment as the baseline. Three controllers are produced, each corresponding to a distinct performance–power trade-off defined by reward parameter λ (HighPerf-CNN, Balanced-CNN, and LowPower-CNN), and deployed within gem5 via ONNX Runtime for runtime inference.

3. To evaluate the CNN controller against the rule-based logic and the fixed-frequency reference under closed-loop simulation. The closed-loop evaluation runs selected PARSEC benchmarks within the full gem5 system.

4. Additionally, an input feature importance analysis is conducted to identify which monitoring signals contribute most to classification accuracy, to reduce implementation overhead.

The expected results of this project include: a labelled dataset on an 8×8 mesh topology; three trained and evaluated CNN controllers; a C++ integration of the DVFS controller within gem5; and a quantitative comparison of the controllers across the selected PARSEC workloads in terms of latency.

1.5 Research Methodology

This thesis uses a simulation-based, supervised-learning methodology. Simulation is the primary environment because no physical many-core hardware was available and because it allows experiments to be repeated under controlled conditions. The platform is gem5 together with its Garnet NoC model, which gives detailed cycle-level router timing and is widely used in academic NoC research.

The methodology proceeds in four phases: data collection from both Garnet Standalone synthetic sweeps and full-system PARSEC runs; supervised training of three CNN controllers under different reward parameters λ, where λ controls the trade-off between throughput and estimated power consumption; ONNX export and integration into gem5; and closed-loop evaluation against the baseline.

This supervised approach extends the earlier rule-based one rather than departing from it. The rule-based controller maps network state to a frequency using rules written by hand. The CNN keeps the same closed-loop structure, but instead of fixed rules it learns the mapping from simulation data.

1.6 Structure of the thesis

The remainder of this thesis is organized as follows. Chapter 2 provides background on Network-on-Chip architecture, DVFS mechanisms, machine learning techniques used in this work, and related prior work on ML-based NoC control. Chapter 3 describes the research methodology, including the simulation platform, data collection strategy, rule-based baseline design, CNN controller design, and evaluation framework. Chapter 4 details the system implementation, covering the gem5/Garnet integration, offline training pipeline, and deployment of the trained models via ONNX Runtime. Chapter 5 presents the results of the CNN training and the closed-loop DVFS evaluation against the rule-based baseline and fixed-frequency reference. Chapter 6 concludes the thesis and outlines directions for future work.

Chapter 2

Background

2.1 Network-on-Chip

2.1.1 Motivation and Architecture

Scaling of Complementary Metal-Oxide-Semiconductor (CMOS) technology has made it possible to place hundreds of processing cores on a single die, producing the many-core System-on-Chip (SoC). As the number of cores grows, the on-chip communication fabric becomes a leading factor in both performance and power. The shared-bus and crossbar interconnects used in earlier multi-core designs do not scale to this regime: a shared bus serializes all traffic and becomes a bottleneck, while a crossbar grows in area and wiring complexity with the number of attached nodes [17].

The NoC was introduced to address this problem [14, 18]. Instead of a single shared medium, a NoC organizes the interconnect as a packet-switched network of routers distributed across the chip and joined by point-to-point links. Each core, cache bank, or memory controller connects to the network through a dedicated Network Interface (NI). Data are divided into fixed-size flow-control units called flits. One or more flits form a packet; its header flit carries the destination address and routing information, and the remaining data flits carry the payload [19]. Many topologies are possible for on-chip networks, including ring, star, mesh, torus, fat-tree, and butterfly, as illustrated in Figure 2.1. The 2-D mesh is the most widely used in practice because of its regular layout, simple wiring, and direct mapping onto a chip floorplan. In a $k \times k$ mesh, each router has up to four directional ports (North, South, East, West) and a local port to the attached Processing Element (PE). Mesh networks are usually combined with Dimension-Order Routing (DOR); under

XY routing a packet completes all of its hops in the X dimension before turning into the Y dimension. XY routing is deterministic, deadlock-free, and cheap to implement in hardware, which makes it the default choice in gem5 [20].

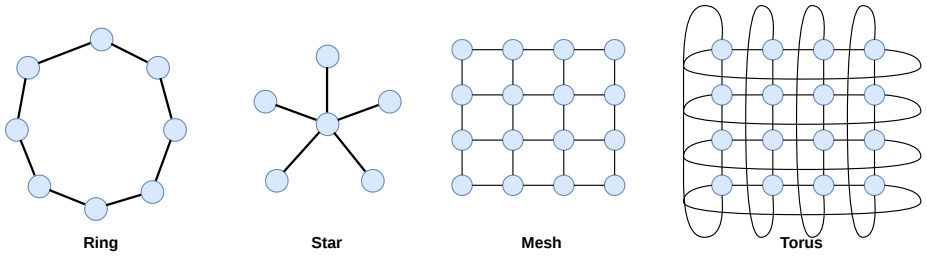


Figure 2.1: Examples of Ring, Star, Mesh, and Torus network structures, reproduced from [21].

Routers use Virtual-Channel (VC) flow control. Each physical link is shared by several virtual channels, each with its own flit buffer. An incoming flit is assigned to a VC, buffered, and forwarded to the next router once a downstream credit indicates free buffer space. By separating the occupancy of a physical channel from the progress of individual packets, VC flow control removes head-of-line blocking and lets packets bound for different outputs share one physical link. Garnet models the full five-stage router pipeline (Buffer Write (BW), Route Compute (RC), Virtual Channel Allocation (VA), Switch Allocation (SA), and Switch Traversal (ST)) at the flit level, which makes it suitable for accurate latency and power estimation [22]. The pipeline organization and the VC structure are the main architectural parameters governing a router's area, power, and throughput.

The 8×8 mesh topology used in this work consists of 64 routers and 64 attached nodes. All 64 routers share a single voltage-frequency domain in the unified DVFS configuration adopted here, simplifying the control problem while still capturing the interaction between network load and energy consumption. This single-domain design represents a practical first step before migrating to per-router or clustered domains, which introduce additional synchronization overhead.

2.1.2 Performance Metrics

This section defines the NoC metrics used throughout the thesis.

Packet latency is the time from the injection of a packet's head flit at the source NI to the reception of its tail flit at the destination NI, measured in network clock cycles. It splits into two parts:

$$L_{\text{total}} = L_{\text{queue}} + L_{\text{network}} \tag{2.1}$$

Queueing latency is the time a packet waits at the source NI for a free VC or output credit before entering the network. Network latency is the time from when the head flit enters the first router to when the tail flit leaves the destination NI, and it includes per-hop pipeline, link-traversal, and serialization delays. At light load, queueing latency is negligible and total latency is set by the fixed traversal delay; at heavy load, queueing latency rises sharply and dominates.

Injected count is the number of packets (or flits) that enter the network at the source NI within an observation window. Received count is the number of packets (or flits) that are delivered at the destination NI within the same window. The difference between the two reflects in-flight traffic currently traversing the network.

Injection rate is the rate at which new packets or flits are presented to the network, in per node per cycle, used as a feature to capture traffic intensity.

Link utilization is the fraction of cycles in which a link carries a flit, averaged over all links. It measures the overall network load. Garnet reports per-link activity factors, which show how evenly traffic is spread across the network.

Hop count is the number of router-to-router links a packet traverses. Under dimension-order routing on a mesh, a packet between (x_s, y_s) and (x_d, y_d) has hop count $|x_d - x_s| + |y_d - y_s|$. The average over delivered packets, `AvgHops`, reflects the spatial distribution of traffic: uniform random traffic gives an average close to half the network diameter, while hotspot or tornado patterns shift it higher.

Operating frequency is the clock rate of the routers and links, and it is the control variable of the DVFS system. A higher frequency raises throughput capacity but increases power, while a lower frequency saves energy at the cost of forwarding rate. The controller's task is to choose the frequency that best balances throughput and energy for the current traffic conditions.

2.2 Dynamic Voltage and Frequency Scaling

DVFS is a power-management technique that varies a circuit's clock frequency and supply voltage at runtime to match the current workload. Its starting point is how dynamic switching power depends on voltage and frequency:

$$P_{\text{dynamic}} \propto \alpha \cdot C \cdot V^2 \cdot f \tag{2.2}$$

where α is the activity factor, C the switched capacitance, V the supply voltage, and f the clock frequency. The activity factor is the term that ties power to the workload. In a NoC router it grows with traffic, because every flit that is buffered or sent across the switch is a switching event, so at a fixed operating point the dynamic power rises and falls with the offered load. Voltage and frequency are not workload signals; they are what the operating point fixes, and the two are coupled: the lowest voltage a circuit can use is bounded by the frequency it must sustain to meet timing. Lowering f lets V drop as well, and since power depends on V^2 the saving is more than linear, whereas pushing f up for performance forces V back up and raises power by the same super-linear margin.

In practice a DVFS-capable component does not vary V and f continuously but runs at a small, fixed set of voltage-frequency pairs, known as operating points or P-states. Each pair is decided at design time: for a given frequency, the lowest voltage at which every timing path still closes under worst-case Process-Voltage-Temperature (PVT) conditions is found and stored in a lookup table. At runtime the controller simply picks one of these points, using the workload signals it observes.

In a NoC router the power that DVFS can act on comes mainly from the input VC buffers and the crossbar switch, with the allocators adding little [23]. The buffers account for most of the static power, while the crossbar draws the most dynamic power under heavy traffic, and that dynamic part scales with $V^2 f$ as in Equation (2.2). This is what DVFS has to work with at the router: dropping voltage and frequency under light traffic cuts power steeply, and raising them again recovers throughput when load returns.

2.3 Multi-core simulation framework

2.3.1 gem5 Simulator

gem5 is an open-source, modular computer-architecture simulator. It supports several instruction set architectures (ARM, x86, MIPS, SPARC, POWER, RISC-V) and a range of Central Processing Unit (CPU) models, from simple in-order timing models to detailed out-of-order pipelines [24]. It grew out of the merger of the M5 [25] and GEMS [26] simulators, combining M5's system-level flexibility with GEMS's detailed memory-hierarchy modelling,

and is now a community-standard tool maintained by academic and industrial contributors.

In gem5, the memory hierarchy is handled by the Ruby subsystem. Ruby implements cache and directory controllers and coherence protocols through the Specification Language for Implementing Cache Coherence (SLICC) domain-specific language, so a protocol can be changed and evaluated without modifying the simulator core [26]. Ruby also owns the on-chip interconnect, which is modelled by Garnet (described in the next section). gem5 advances time in discrete ticks; by default one tick is one picosecond, giving a timing resolution of 1 ps.

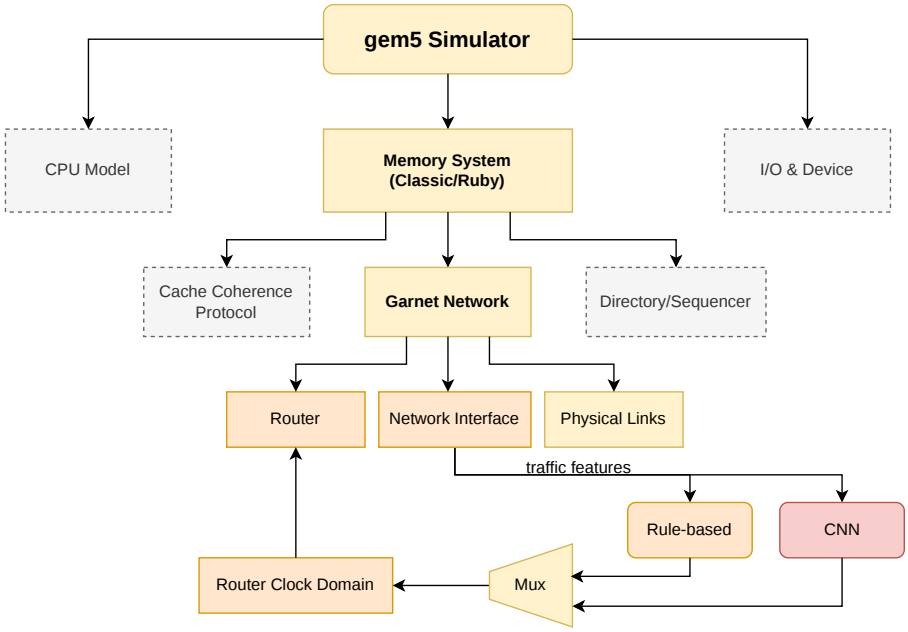


Figure 2.2: Architecture of the gem5 simulator

2.3.2 Garnet NoC Model

Garnet is the detailed on-chip-network model inside gem5 [22]. It is cycle-accurate, implementing the full five-stage router pipeline and virtual-channel flow control at the flit level. Each router, link, and network interface is a separate C++ object with its own clock domain, which makes it straightforward to assign per-component latencies and to scale the network clock independently of the CPU clock.

In Garnet standalone mode, a lightweight dummy coherence protocol replaces the full Ruby protocol, and a built-in synthetic traffic generator injects packets directly into each node's network interface at a configurable rate. No real CPUs, caches, or memory controllers are instantiated, which allows fast sweeps of the injection-rate versus latency space for a given traffic pattern and frequency.

2.3.3 Full-System Simulation and PARSEC Benchmarks

For full-system simulation, gem5 boots a real Linux kernel and runs unmodified application binaries, capturing how the application interacts with the operating system, which subjects the network to the irregular, time-varying traffic of actual workloads. To produce such traffic, this work uses the Princeton Application Repository for Shared-Memory Computers (PARSEC) suite.

PARSEC is a widely used benchmark suite for characterizing Chip-Multiprocessor (CMP) workloads [27]. Its applications come from several domains, including financial analytics, scientific simulation, image processing, and data clustering. What makes the suite useful here is that they differ widely in how much their threads communicate and share data, and therefore in how heavily they load the on-chip interconnect: some are almost entirely compute-bound, while others are dominated by communication.

Each application can be run at three input sizes, listed in Table 2.1. The input set fixes the problem size and, with it, both the volume of network traffic and the simulation time: `simsmall` keeps runs short for quick experiments, whereas `simlarge` generates enough traffic to exercise the network realistically. The sizes used in this work are given in Chapter 3.

The workloads also differ in the kind of pressure they place on the system, which Table 2.2 summarizes for the large input sets. Each one is classified along two axes: whether it is computation- or communication-intensive, and whether its shared-data access is intensive or not. The extremes are easy to read off: blackscholes is almost purely compute-bound with little sharing, whereas canneal and ferret are communication-intensive with heavy data sharing [28]. For a study of NoC behavior, the communication-intensive workloads are the most informative, since they generate the sustained, bursty interconnect traffic that a DVFS controller has to respond to.

Table 2.1: Work units contained in PARSEC simulation input sets for each benchmark, adapted from [27].

Program	Input Set		
	simsmall	**simmedium**	**simlarge**
blackscholes	4,096	16,384	65,536
bodytrack	60	60	60
canneal	$\leq 5.0 \times 10^4$	$\leq 1.0 \times 10^5$	$\leq 2.0 \times 10^5$
dedup	2,841	8,108	94,130
facesim	80,598	80,598	80,598
ferret	16	64	256
fluidanimate	$\leq 3.5 \times 10^4$	$\leq 1.0 \times 10^5$	$\leq 3.0 \times 10^5$
freqmine	23	46	91
raytrace	1,980	8,040	32,400
streamcluster	4,096	8,192	16,384
swaptions	16	32	64
vips	475 (50)	1369 (74)	3612 (84)
x264	8	32	128

Table 2.2: PARSEC benchmark characterization with large input sets, adapted from [28].

Workload	Problem size (all large input sets)	C1-I1	C1-I2	C2-I1	C2-I2	Sum
blackscholes	65,536 options	98.1%	0.0%	0.0%	0.0%	98.1%
bodytrack	80,598 particles, 372,126 tetrahedra	9.3%	83.1%	2.1%	3.3%	97.8%
canneal	4 frames, 4,000 particles	9.3%	7.9%	1.4%	76.5%	95.1%
dedup	256 queries, 34,973 images	8.8%	72.3%	9.2%	7.6%	97.9%
facesim	990,000 click streams	1.4%	2.0%	86.5%	6.7%	96.6%
ferret	400,000 netlist elements	0.8%	4.7%	7.3%	84.7%	97.5%
fluidanimate	16,384 points, 128 point dimension	12.8%	5.6%	75.6%	1.5%	95.5%
freqmine	5 frames, 300,000 particles	4.3%	73.3%	10.7%	5.6%	93.9%
streamcluster	64 swaps, 20,000 simulations	8.1%	8.6%	69.4%	10.4%	96.5%
swaptions	2,662×5,500 pixels	17.4%	0.0%	80.9%	0.0%	98.3%
vips	128 frames, 640×360 pixels	0.0%	9.1%	75.2%	14.7%	99.0%
x264	184MB data	4.6%	3.0%	3.2%	84.7%	95.5%

C1-I1: Computation intensive LW with non-intensive shared-data access.
C1-I2: Computation intensive LW with intensive shared-data access.
C2-I1: Communication intensive LW with non-intensive shared-data access.
C2-I2: Communication intensive LW with intensive shared-data access.

2.4 ML in DVFS

2.4.1 One-Dimensional Convolutional Neural Network

A One-dimensional Convolutional Neural Network (1D-CNN) uses convolutional layers to extract features and fully connected layers to classify them. Each convolutional layer applies a set of filters that slide along the sequence dimension, and because a filter shares its weights across all positions, it responds to a given local pattern wherever that pattern occurs in the input [29]. This is useful for the input used in this work, a short window of consecutive NoC traffic statistics: what distinguishes one traffic regime from another lies in how the values change across the window rather than in any single sample, and convolution is able to capture that local structure [30].

A `Conv1d` layer slides C_{out} learned filters of width k over an input of shape (C_{in}, L), where C_{in} is the number of input channels and L the sequence length:

$$z_{c,t} = \sum_{i=1}^{C_{\text{in}}} \sum_{j=0}^{k-1} w_{c,i,j}\, x_{i,t+j} + b_c, \tag{2.3}$$

The shared weights $w_{c,i,j}$ are applied at every position t, so one filter detects the same feature across the whole window, and stacking several convolutional layers lets the network combine these features into more abstract ones. The feature maps are then flattened and passed through fully connected layers with ReLU activations [31], and a final softmax layer turns the outputs into a probability distribution over the K classes. Each class is one discrete V/F operating point, so the predicted class is the frequency level the controller selects for the current network state.

2.4.2 Model Export and Runtime Inference

The controller is trained offline in PyTorch, but it has to run online inside gem5, whose core is written in C++ and does not embed a Python interpreter. The model therefore cannot simply be invoked through PyTorch during simulation; it must be turned into a self-contained component that the C++ code can load and evaluate on its own. This is the main practical obstacle to deploying a learned controller in an embedded environment, and it is the problem the Open Neural Network Exchange (ONNX) format is designed to solve [32].

The most direct alternative is to use the PyTorch C++ runtime, such as LibTorch or TorchScript [33, 34]. This would remove the need for a Python

interpreter, but the simulator would still be tied to the PyTorch runtime. ONNX keeps PyTorch only in the offline training and export stage, while the online simulator only needs a normal C++ inference library. This also explains why the current implementation uses the CPU execution provider rather than a GPU backend. The controller is small, and each decision only requires one forward pass, so CPU inference is sufficient for the gem5 experiments. Using a GPU would introduce extra dependencies and synchronization overhead without a clear benefit in this setting. At the same time, the ONNX format does not make the model CPU-specific: ONNX Runtime can execute the same exported graph through different execution providers, including CPU, CUDA, and TensorRT [35]. Thus, ONNX is selected mainly to make the trained controller portable from PyTorch to gem5, while still leaving open the possibility of using another backend in future implementations.

ONNX stores a trained model as a framework-independent computational graph. Its nodes are operators such as convolution, matrix multiplication, and activation functions, drawn from a versioned operator set; its edges carry tensors; and the learned weights are held in the graph as constant initializers. Because the operator set fixes the meaning of each node, the resulting file is self-contained: a model exported from PyTorch can be read and evaluated by any compliant runtime, regardless of the framework or library versions used to train it [36]. PyTorch writes this file with `torch.onnx.export`, which runs the model once on a sample input, records the operations executed during that forward pass, and saves them together with the weights as an `.onnx` file. Since the trace is taken at the input size the controller will actually use, no shape inference is required at inference time.

The exported graph is executed by ONNX Runtime (ORT), a C++ inference engine [37]. The model is loaded once when the simulator starts up, into an `Ort::Session`. At this point ORT optimizes the graph, folding constants and fusing operators, and binds it to an execution provider, the CPU backend, although the same file could run on CUDA or TensorRT without being re-exported. Every subsequent prediction reuses this session: the current feature window is passed in as an `Ort::Value` wrapping the existing C++ buffer, `session.Run` evaluates the network, and the output logits are reduced with `argmax` to give the selected V/F level. Keeping one session for the whole run means each decision costs only the forward pass.

This setup suits the controller well. Inference is deterministic, so repeated simulation runs remain reproducible; ORT links in as a self-contained C++ library, integrating into gem5 as a single additional dependency; and because each decision is one forward pass through a small network, its cost is

negligible relative to the surrounding simulation, with the stored weights occupying little memory. The same `.onnx` file is also what many on-chip inference accelerators and quantization toolchains expect as input: deep-learning compilers such as TVM, for example, ingest ONNX models and compile them to CPUs, GPUs, and specialized accelerators [38]. Although the controller here runs in simulation, the export route it uses is therefore the same one that a hardware implementation would follow.

2.5 Related work area

2.5.1 Rule-Based Flow Control

Traditional NoC mechanisms relied on simple, rule-based control with fixed thresholds. For source-end regulation, Ogras and Marculescu [39] predict future network congestion and adjust the packet injection rate at traffic sources to bound the number of in-flight packets, outperforming traditional reactive regulation on both synthetic and real traffic. Nychis et al. [40] instead apply application-aware source throttling, using a simple threshold rule on per-node injection-queue intensity to decide which cores to throttle, and show that throttling the right applications is essential to realizing application-level throughput gains. Beyond source-end regulation, Soteriou and Peh [41] reduce link power by switching links on and off according to hardware-counter traffic measurements, accepting a wake-up penalty in exchange for power savings during idle periods.

The same rule-based philosophy underlies much early DVFS control. Mishra et al. [16] tune router frequency in response to network load through three threshold-driven techniques (FreqBoost, FreqThrtl, and FreqTune), reducing average latency or improving energy-delay product depending on the policy, evaluated on an 8×8 wormhole mesh. Chen et al. [42] treat the NoC and shared last-level cache as a single voltage/frequency domain, monitor average memory access time, and drive DVFS with a PID controller, reporting 33% dynamic-energy savings on PARSEC; their work also reports a naive rule-based DVFS policy that maps load levels directly to frequencies as a comparison point. In a similar threshold-driven spirit, Hesse and Jerger [43] adjust router frequency from cache-coherence event prediction, scaling voltage and frequency ahead of the traffic those events generate.

Yao et al. [44] proposed a fuzzy flow regulation system for NoC traffic management, addressing the limitations of static leaky-bucket regulators that cannot adapt to changing network conditions. The system monitors average

packet latency and link utilization, mapping them to a saturation factor K via center-of-gravity defuzzification. A fuzzy controller then uses K, the current injection rate ρ_0, and a pre-computed reference rate ρ_r to modulate traffic through a seven-rule linguistic policy. Evaluated in GEMS on a 4-chip mesh CMP, the system achieves up to 41% throughput improvement over static regulation.

Despite this effectiveness, the approach relies on hand-crafted membership functions and rule tables calibrated for specific topologies, with the reference rate derived under idealized zero-delay assumptions. These constraints limit transferability to unseen workloads, motivating a data-driven alternative that learns the control mapping directly from empirical observations.

2.5.2 AI-Based NoC Control

Zhang et al. [45] survey ML-based NoC research across two directions: performance modeling and component-level design optimization, including routing, arbitration, and power management. DVFS control is among the most actively studied problems. On the design-optimization side identified by that survey, Lin et al. [46] use deep reinforcement learning with a Monte Carlo tree search to place loops in a routerless NoC, showing that learning is effective for NoC design and not only for runtime control.

Early learned controllers build on classical control-theoretic methods. Won et al. [47] use a Proportional-Integral controller to bootstrap the online training of an artificial neural network for uncore (NoC and last-level cache) DVFS. Because the PI controller supplies the training signal at runtime, the method requires no pre-recorded training data and improves the uncore energy-delay product by 27% on full-system PARSEC simulation. Fettes et al. [48] combine supervised prediction with reinforcement learning: three supervised models predict buffer utilization, its variance, and power-throughput metrics, and an RL agent uses these predictions to select among discrete DVFS modes. They also evaluate a ridge-regression baseline that predicts buffer utilization directly for per-router frequency selection. Zheng and Louri [49] use a Q-learning agent over cache- and network-level features and replace the Q-table with a compact offline-trained ANN, reducing area overhead by 67%. Clark et al. [50] build on the same power-gating-and-DVFS combination with a single-inductor multiple-output voltage regulator, and use a machine-learning model that predicts near-future traffic load to select each router's mode proactively.

On an 8×8 mesh in gem5/Garnet with PARSEC benchmarks, their design

achieves a 26% power reduction and a 7% performance improvement. For traffic admission control, Wang et al. [51] use a centralized ANN-AC that compresses 256-dimensional per-node state into 12 aggregated features; it reaches 90% prediction accuracy and reduces execution time by 17.8% relative to an unregulated baseline. Yin et al. [52] apply reinforcement learning to NoC arbitration and analyze the learned network weights to distill a lightweight, hardware-friendly policy that remains effective under heavy contention.

These works establish three principles relevant to the present work: offline training on simulation data is practical and effective; windowed feature aggregation is critical for capturing temporal dynamics; and supervised classification can match RL in settings where control objectives can be expressed through labeled examples.

Chapter 3

Methods

3.1 Research Process and Research Paradigm

An overview of the end-to-end research workflow is presented as an activity diagram.

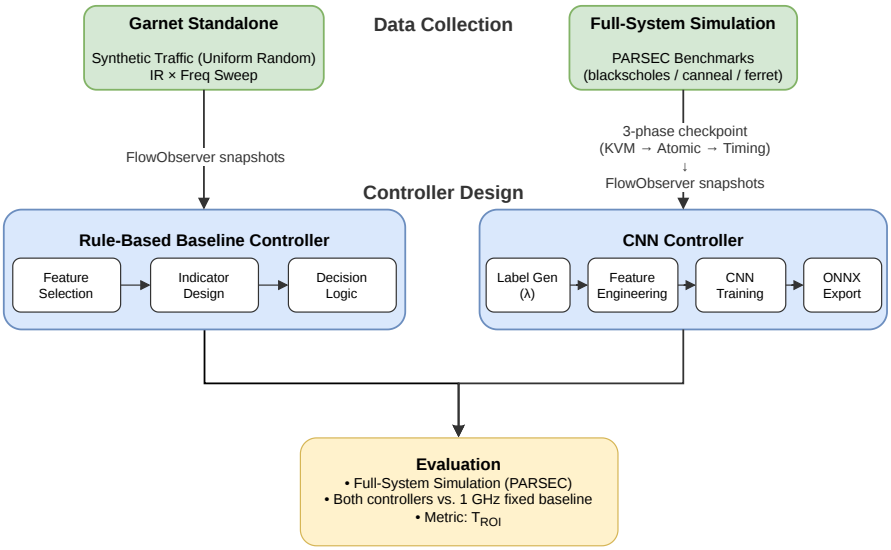


Figure 3.1: Research workflow: controller design and evaluation pipeline

The study uses two simulation environments. Garnet Standalone is used to characterize network behavior across a wide range of injection rates and frequency operating points; this characterization informs which network statistics the rule-based baseline acts on and how its congestion indicators are structured. Full-system simulation of a communication-intensive benchmark

workload then provides the feature distributions used to set the rule-based thresholds, as well as the labelled data used to train the CNN controllers. Both controllers are evaluated in the full-system environment against a fixed-frequency 1 GHz reference, using benchmarks that differ in communication intensity.

3.2 Simulation Platform

3.2.1 Test Environment

All simulations are conducted on a departmental CPU server running Ubuntu 22.04. gem5 is built within a Docker container environment, which isolates the complex compilation dependencies gem5 requires, including specific compiler versions and Python libraries.

The two simulation modes operate under different execution environments. Garnet Standalone simulations are run directly within the Docker container, where the full gem5 runtime environment is self-contained. Full-system simulations are launched from the host, since they need system-level resources such as KVM hardware virtualisation for accelerated boot, which the container does not provide.

3.2.2 NoC Simulation Configuration

An 8×8 mesh topology is used in all experiments. A single unified DVFS domain is assumed, so all routers and links operate at the same frequency at any given time. This follows common practice in NoC-level DVFS work and avoids inter-domain synchronisation, which this study does not address.

Frequency scaling is implemented through buffer stall injection instead of true clock-domain switching. The base ruby-clock runs at 10 GHz throughout, and an effective frequency reduction is obtained by inserting stall cycles at the network interface. This lets frequency vary programmatically within a single run, without gem5 having to manage multiple clock domains at runtime.

3.3 Data Collection

3.3.1 Synthetic Traffic Simulation

Garnet Standalone simulation provides a controlled environment for sweeping network conditions systematically, isolating the NoC from full-system

complexity, and the injection rate and frequency operating point can be varied independently over a wide parameter space. This mode is used to characterize the behavior of the network under synthetic load. The characterization informs the structure of the rule-based baseline, namely which network statistics it observes and how its congestion indicators are formed, while the concrete threshold values are set from the full-system feature distributions.

The synthetic traffic pattern is uniform random. Although this does not reproduce every spatial characteristic of real workloads, it provides a clean and reproducible basis for exploring the relationship between injection load, frequency, and network behavior across a systematic sweep of parameters.

Data collection follows a windowed observation structure. Each simulation run is divided into consecutive time windows, within each of which the injection rate varies linearly from a specified start value to an end value, and a fixed frequency operating point is assigned.

To ensure broad coverage of traffic dynamics, six injection trajectories are defined, each specifying a continuous sequence of start and end injection rates across the windows. The trajectories are designed to represent qualitatively distinct load patterns, including gradual ramps, single-peak humps, valleys, and sustained high-load segments, so that the characterization covers a variety of temporal injection dynamics rather than a single regime.

3.3.2 Full-System Simulation

Full-system simulation is used to collect network traffic data under realistic application-driven workloads. Three PARSEC benchmarks are selected to cover a range of communication characteristics, as summarized in Table 2.2. Blackscholes is predominantly compute-intensive with minimal inter-node communication, while canneal and ferret are communication-intensive with irregular shared-data access patterns. This selection provides both a low-communication setting, where the benefit of frequency scaling is expected to be limited, and high-communication settings, where the controller has meaningful decisions to make.

To avoid repeating the expensive OS boot and benchmark initialisation for every configuration, full-system simulation uses a three-phase checkpoint strategy. The first phase uses KVM-accelerated execution to fast-forward to a point near the Region of Interest (ROI) boundary; the second phase switches to AtomicSimpleCPU to locate the exact ROI entry tick reproducibly; the third phase restores from that checkpoint under TimingSimpleCPU for NoC data collection. This strategy ensures all experimental runs start from an identical

system state while keeping the per-run simulation cost manageable.

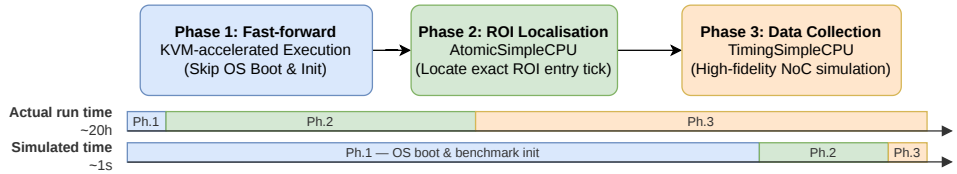


Figure 3.2: Three-phase checkpoint strategy for full-system gem5 simulation.

3.4 Rule-Based Controller

A rule-based DVFS controller makes a useful controller, which requires no training. The design draws on both simulation environments: the Garnet Standalone characterization determines which statistics the rules act on and how the congestion indicators are structured, while the concrete threshold values are fixed from the full-system feature distributions, so that they match the operating ranges seen under real workloads.

3.4.1 Choice of Input Features

The `FlowObserver` module exposes a range of network-level statistics at the end of each monitoring window, including total and network latency, queueing latency, hop count, injection rate, link utilization, and aggregate flit counters. Effective features for the rule must (a) carry information about the current congestion state, (b) react quickly enough to inform the frequency decision in the next window, and (c) remain meaningful across the traffic patterns the network experiences.

Three categories of features are used. The first category consists of latency features, specifically total latency and network latency, which reflect the current congestion state. Total latency aggregates queueing and traversal delay, while network latency isolates contention along the routing path. The second category consists of traffic-volume features, namely injection rate and link utilization, which capture the load placed on the interconnect and respond before queueing effects fully manifest as elevated latency. The third category consists of temporal features derived by comparing the current observation x_t against the previous window x_{t-1}, which allow the rule to respond to trends rather than reacting solely to the instantaneous network state.

3.4.2 Indicator Design

The rule combines three congestion indicators computed from the feature vector $\{x_{t-1}, x_t\}$ over two consecutive monitoring windows.

Absolute latency level. Both average total latency $\mathrm{TotLat}(x_t)$ and average network latency $\mathrm{NetLat}(x_t)$ are thresholded at two levels to mark congestion onset and saturation respectively. Using both measures avoids blind spots, as the ratio of queueing to network latency varies across traffic patterns.

Latency trend. A relative rise in either latency signal between x_{t-1} and x_t indicates building congestion before absolute thresholds are crossed. Total and network latency are treated asymmetrically because their noise characteristics differ.

Link saturation. Injection rate and average link utilization jointly indicate that the network is approaching capacity even before latency rises. A heavy-load condition requires both signals to exceed their thresholds simultaneously; a moderate-load condition fires when either does, but is only acted upon in conjunction with the trend signal.

The target frequency is selected by a three-tier priority scheme:

- **2.0 GHz** if any severe condition is active—absolute latency at its severe level, or heavy load detected.
- **1.5 GHz** if an elevated condition is active—latency above its elevated threshold, or latency rising under moderate load.
- **1.0 GHz** otherwise.

3.5 CNN-Based DVFS Controller

3.5.1 Label Generation

Each training sample requires a ground-truth label indicating which of the three frequency levels would have been the most beneficial choice at the moment a DVFS decision is made. Rather than encoding this choice as a hand-tuned threshold rule, the label is derived from a reward function that jointly accounts for throughput gain and power cost. This formulation reduces the label design problem to selecting a single trade-off parameter, and makes the relationship between that parameter and the resulting controller behaviour explicit.

For each candidate frequency $f \in \{1.0, 1.5, 2.0\}$ GHz, a reward is computed as:

$$R(f) = r_{\text{tp}}(f) - \lambda \cdot c(f) \tag{3.1}$$

where $r_{\text{tp}}(f) = \Delta_{\text{flits}}(f)/\Delta_{\text{flits}}(1\,\text{GHz})$ is the throughput ratio relative to the 1 GHz baseline, $c(f) = (f/1\,\text{GHz})^2$ is a simplified estimate of power cost, and λ controls the trade-off between the two.

Frequencies that worsen normalised latency beyond a tolerance of 3% relative to the 1 GHz baseline are excluded. The ground-truth label is then the frequency with the highest reward among the remaining candidates.

The choice of λ directly controls the trade-off between performance and energy efficiency, and different values lead to qualitatively different controllers. To select appropriate values systematically, λ is swept over a range of candidate values. For each value, labels are generated and a model is trained and evaluated under 3-fold cross-validation. The resulting label distributions and balanced accuracy scores are examined together to identify values that produce well-separated, representative label distributions without sacrificing classification performance.

Three values are selected to represent distinct operating priorities:

- **HighPerf-CNN**: the low penalty on power cost results in a near-balanced label distribution across all three frequency classes, and the model achieves the highest and most stable balanced accuracy across folds. This variant is chosen as the primary result.
- **Balanced-CNN**: an intermediate penalty that moderates high-frequency selection while still allowing the controller to respond to genuine throughput opportunities.
- **LowPower-CNN**: a strong penalty that results in a label distribution heavily skewed towards 1.0 GHz. The controller is conservative by design and elevates frequency only under sustained high load.

3.5.2 Feature Engineering and Normalisation

A single instantaneous NoC snapshot is insufficient to distinguish between a network that is building towards saturation and one that has already passed its congestion peak. Each sample is represented as a flattened sliding window of $H = 2$ consecutive `FlowObserver` snapshots immediately preceding the decision point, giving the model access to both the current state and its immediate predecessor.

The 14 raw NoC metrics recorded per snapshot are listed in Table 3.1. The metrics are selected to provide complementary coverage of three aspects of network state: latency (average total, network, and queueing latency at both the packet and flit granularity), throughput (injected and received deltas for both packets and flits), and load (link utilization and injection rate). These dimensions characterize whether the network is under-utilized, approaching saturation, or already congested. No handcrafted derived features are constructed; the raw signals are supplied directly, allowing the model to learn which combinations are most predictive without engineering assumptions about which interactions matter.

Table 3.1: Per-window raw feature set ($H = 2$, total input dimension $= 28$).

Feature	**Description**
`AvgTotalLatency_pkt`	Average total packet latency (cycles)
`AvgNetworkLatency_pkt`	Average in-network packet latency (cycles)
`AvgQueueingLatency_pkt`	Average queueing latency per packet (cycles)
`AvgHops`	Average hop count per packet
`PacketsInjectedDelta`	Packets injected in the observation window
`PacketsReceivedDelta`	Packets received in the observation window
`AvgLinkUtil`	Average link utilisation
`InjectionRate`	Packet injection rate (packets per node per cycle)
`FlitsInjectedDelta`	Flits injected in the observation window
`FlitsReceivedDelta`	Flits received in the observation window
`FlitInjectionRate`	Normalised flit injection rate
`AvgNetworkLatency_flit`	Average in-network flit latency (cycles)
`AvgQueueingLatency_flit`	Average queueing latency per flit (cycles)
`AvgTotalLatency_flit`	Average total flit latency (cycles)

Raw NoC metrics span several orders of magnitude: latency values may reach thousands of cycles while link utilisation lies in $[0, 1]$. Without normalisation, gradient-based optimisation is dominated by high-magnitude features, effectively ignoring low-magnitude ones. All features are therefore standardised to zero mean and unit variance using z-score normalisation:

$$\hat{x}_i = \frac{x_i - \mu_i}{\sigma_i} \tag{3.2}$$

where μ_i and σ_i are estimated from the training split only, preventing any statistical leakage from validation data into the scaler. The same parameters are embedded into the gem5 inference component so that runtime inputs are normalised identically to training inputs.

3.5.3 Model Architecture and Training

The DVFS classifier adopts a CNN-based architecture in which a one-dimensional convolutional front-end processes the temporal history, followed by a fully connected head that produces frequency-class logits. The motivation for the convolutional component is that patterns relevant to congestion prediction, such as a rapid increase in queueing latency over successive windows, are more naturally captured by filters that slide over the time axis than by a plain MLP that treats all inputs symmetrically.

The 28-dimensional input vector is reshaped into a $(B \times F \times H)$ tensor, where $F = 14$ is the feature dimension and $H = 2$ the time dimension, so that the convolution operates along the time axis. Each filter spans all 14 features within its two-window kernel, with weights shared across the temporal axis. This gives the model a structural inductive bias toward how the metrics change between consecutive windows, rather than treating each of the 28 inputs as an independent dimension as a fully connected layer would.

1. Conv1d($14 \to 32$, kernel 2, padding 1) $\to$ BatchNorm $\to$ ReLU
2. Conv1d($32 \to 64$, kernel 2, padding 0) $\to$ BatchNorm $\to$ ReLU

The resulting feature map is flattened and passed to the fully connected head:

1. Linear($128 \to 20$) $\to$ ReLU $\to$ Dropout($p = 0.3$)
2. Linear($20 \to 3$) [raw logits]

The hidden layer width is chosen to provide sufficient capacity to represent the decision boundaries while keeping the parameter count low enough to avoid overfitting. Dropout is applied after the hidden layer activation to promote generalisation to minority-class samples.

The NoC spends the majority of its time in lightly loaded states where 1.0 GHz is the optimal choice, so the label distribution is inherently skewed. To counteract this, per-class weights are computed as

$$w_c = \frac{N_{\text{train}}}{K \cdot N_c} \tag{3.3}$$

where N_{train} is the total number of training samples, $K = 3$ the number of classes, and N_c the count of class c. These weights scale the cross-entropy loss so that misclassifying a rare high-frequency sample incurs proportionally

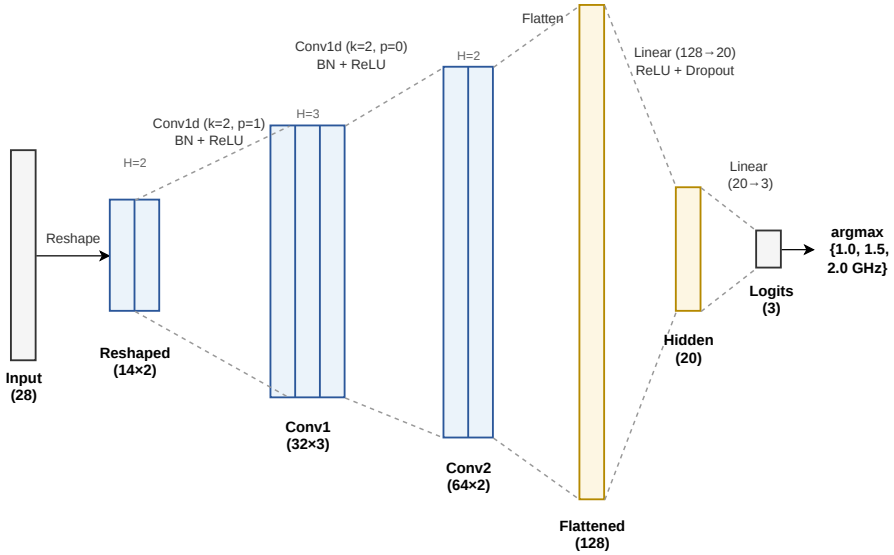


Figure 3.3: Architecture of the CNN controller.

greater penalty. Model selection is based on balanced accuracy rather than raw accuracy, which treats each class recall equally and is a more meaningful metric under class imbalance.

Given the limited number of training samples obtainable from full-system simulation, a single train/validation split carries high variance in the resulting performance estimate. Stratified k-fold cross-validation is therefore adopted: the dataset is partitioned into folds with preserved class proportions, and each fold serves as the validation set once while the remaining folds are used for training. The model is trained using the Adam optimiser with a step-decay learning rate schedule. The fold with the highest balanced validation accuracy is selected as the final model.

3.5.4 Ablation Study Design

Two ablation experiments are conducted to validate the two principal design choices in the feature representation: the composition of the 14-feature input set and the length of the historical context window. Both experiments use the same CNN architecture, training procedure, and 3-fold stratified cross-validation protocol described in Section 3.5.3, so that any change in balanced accuracy can be attributed solely to the input configuration under study.

3.5.4.1 Feature Group Ablation

The 14 features are partitioned into seven semantically coherent groups according to the aspect of network state they capture (Table 3.2). The performance drop relative to the full-feature baseline quantifies how much each group contributes to classification. A positive drop indicates that the group carries information the model relies on; a drop near zero suggests the remaining features already encode the same signal.

Table 3.2: Feature groups used in the ablation study.

Group	Features	Count
Latency (pkt)	`AvgTotalLatency_pkt`, `AvgNetworkLatency_pkt`, `AvgQueueingLatency_pkt`	3
Latency (flit)	`AvgNetworkLatency_flit`, `AvgQueueingLatency_flit`, `AvgTotalLatency_flit`	3
Injection rate	`InjectionRate`, `FlitInjectionRate`	2
Flits	`FlitsInjectedDelta`, `FlitsReceivedDelta`, `FlitInjectionRate`	3
Packets	`PacketsInjectedDelta`, `PacketsReceivedDelta`	2
Hops	`AvgHops`	1
Link util.	`AvgLinkUtil`	1

3.5.4.2 History Window Ablation

The baseline representation concatenates two consecutive snapshots ($H = 2$), providing the model with both the current network state and its immediate predecessor. To assess whether this temporal context genuinely improves classification, a reduced variant with $H = 1$ is evaluated. When $H = 1$, only the snapshot immediately preceding the decision point is retained, so the model must predict the optimal frequency from a single instantaneous observation. The difference in balanced accuracy between the two settings indicates how much predictive value the additional historical window contributes.

3.5.5 Deployment in gem5

Deploying a trained model inside a C++ simulation environment requires a self-contained inference path that introduces no Python runtime dependency. The ONNX interchange format serves this role: the selected checkpoint is exported to ONNX, allowing the gem5 component to issue single-sample queries at runtime via the ONNX Runtime C++ API. Prior to deployment, numerical consistency between the PyTorch and ONNX representations is verified to ensure that the frequency decision is identical under both runtimes.

A critical correctness requirement is that the feature vector assembled at runtime must be normalised by exactly the same parameters used during training. The scaler parameters are compiled directly into the gem5 binary, eliminating any possibility of a runtime mismatch between the Python training pipeline and the C++ inference path.

At each DVFS decision point, the `FlowRegulator` retrieves the two most recent `FlowObserver` snapshots, assembles the 28-dimensional feature vector, applies z-score normalisation, and queries the ONNX Runtime session. The predicted class index is mapped to a target frequency, which is enforced in the network interface by adjusting the per-flit stall count. This closed-loop control cycle repeats at every observation window boundary throughout the simulation.

3.6 Evaluation Framework

Controller performance is assessed against a fixed-frequency baseline running at 1 GHz throughout the simulation. The primary evaluation metric is the elapsed simulation time within the ROI, defined as

$$T_{\mathrm{ROI}} = t_{\mathrm{leave}} - t_{\mathrm{enter}} \tag{3.4}$$

where t_{enter} and t_{leave} are the tick values logged when the PARSEC hooks print `[HOOKS] Entering ROI` and `[HOOKS] Leaving ROI` to the simulated console. Because ticks measure absolute simulation time rather than CPU cycles, T_{ROI} can be compared directly across runs at different frequencies without normalization.

The controller is considered effective if it achieves a lower T_{ROI} than the 1 GHz baseline when the workload is communication-intensive enough to benefit from higher frequencies, and a comparable T_{ROI} at reduced average frequency when it is not.

Chapter 4

System Implementation

4.1 Overall System Architecture

Figure 4.1 illustrates the overall architecture of the DVFS control system. The design embeds a learned frequency controller directly into the gem5 simulator, enabling closed-loop, cycle-accurate DVFS decisions during simulation without modifying the underlying hardware model.

The system operates across two distinct phases. **Offline**, a Python training pipeline processes gem5 trajectory data to train a CNN that maps observed network conditions to an optimal router frequency class. The trained model is exported in ONNX format, and the corresponding normalization parameters are serialized into a C++ header file (`cnn_scaler_params.hh`).

Online, the control loop runs inside gem5 as follows. Every 20 M ticks, `FlowObserver` samples 14 network metrics from the 8×8 Garnet mesh and assembles them into a 14×2 matrix capturing two consecutive observation windows (`hist0` and `hist1`). This matrix is passed to `FlowRegulator`, which hosts the `CNNController`: the raw features are z-score-normalized using the pre-compiled scaler parameters, arranged into a 14×2 tensor (14 features over two observation windows), and forwarded through two 1-D convolutional layers followed by two fully-connected layers, producing a frequency class via `argmax`. `GarnetNetwork` then applies this frequency by adjusting the router stride via `setStrideFromFreq()`, completing the control loop.

This architecture cleanly separates the learning problem from the simulation infrastructure: CNN is trained entirely offline on pre-collected trajectories and integrated into gem5 as a static artifact, while the online components handle only lightweight inference and frequency application at

runtime.

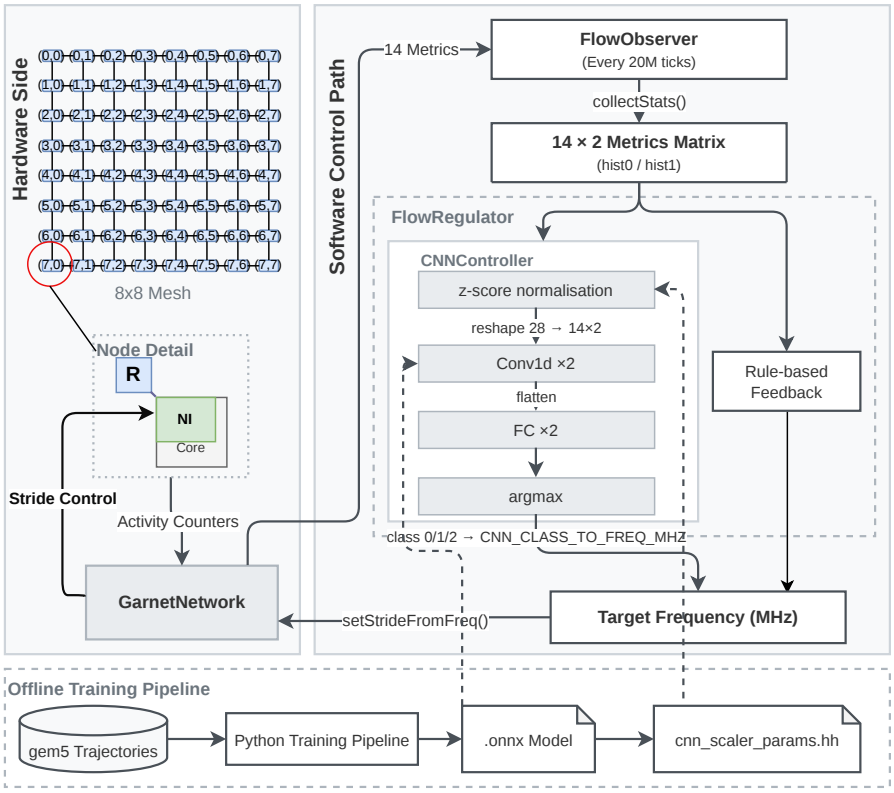


Figure 4.1: Overall architecture of the CNN-based DVFS control system.

4.2 Runtime Integration in gem5

4.2.1 Clock Domain and Frequency Scaling

The simulation employs a single 10 GHz system clock (`clockPeriod() = 100 ticks`), shared by all `ClockedObject` instances. DVFS is therefore implemented as logical wakeup spacing rather than physical clock switching: directly modifying `clockPeriod()` at runtime would misalign already-scheduled events with the new period, causing assertion failures.

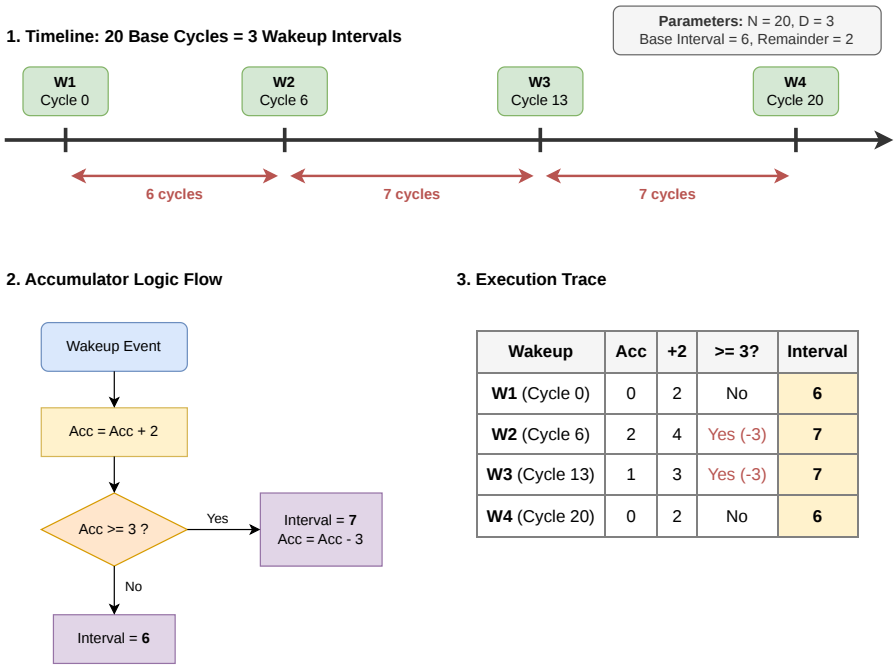
The effective network frequency is emulated by inserting idle cycles between NI wakeups. For a target frequency f (in GHz), rounded to the nearest 0.1 GHz:

$$f_{\text{int}} = \text{round}(f_{\text{GHz}} \times 10) \tag{4.1}$$

the wakeup interval is reduced by the GCD of the base and target:

$$g = \gcd(100, f_{\text{int}}), \qquad N = \frac{100}{g}, \qquad D = \frac{f_{\text{int}}}{g} \tag{4.2}$$

Non-integer intervals (e.g. $N/D = 20/3$ at 1.5 GHz) are handled by `advanceStride()`, which maintains an error accumulator initialized to zero: each wakeup adds N mod D; when the accumulator reaches D, the interval extends by one cycle, yielding a repeating pattern such as $[6, 7, 7, \ldots]$. Early wakeups are suppressed by rescheduling to the next valid cycle boundary without advancing the accumulator. On each DVFS transition, `setStrideFromFreq()` updates the wakeup interval and resets all NI accumulators.



Wakeup	Acc	+2	>= 3?	Interval
W1 (Cycle 0)	0	2	No	**6**
W2 (Cycle 6)	2	4	Yes (-3)	**7**
W3 (Cycle 13)	1	3	Yes (-3)	**7**
W4 (Cycle 20)	0	2	No	**6**

Figure 4.2: Logical Wakeup Spacing (1.5 GHz on 10 GHz Base).

4.2.2 Network Observation: `FlowObserver`

`FlowObserver` is a gem5 event owned by `GarnetNetwork` that orchestrates the closed-loop control path. Every 20,000,000 ticks it computes per-window delta statistics, invokes `FlowRegulator` for a frequency decision, applies the result via `setStrideFromFreq()`, and optionally writes a CSV row. The interval is expressed in ticks so that observation windows remain wall-clock aligned across all DVFS levels. All 14 per-window metrics are computed as deltas over the observation window rather than cumulative totals.

`FlowObserver` supports four mutually exclusive operating modes, selected via command-line parameters and summarized in Table 4.1.

Table 4.1: FlowObserver operating modes.

Mode	Command-line flag	Control path
CNN control	`-cnn-model-path=...`	`cnnInference()`
Rule-based	(no model flag)	`RuleDecision()`
Frequency schedule	`-freq-schedule=...`	pre-defined per-window list
Disabled	`-no-freq-control`	no frequency switching

CNN control is the primary evaluation mode. Rule-based mode serves as a fallback when no model file is supplied. Frequency schedule mode drives controlled frequency transitions during training data collection (Section 4.2.4). Disabled mode produces fixed-frequency baseline runs for evaluation.

4.2.3 Frequency Decision: `FlowRegulator`

`FlowRegulator` translates per-window network metrics into a target frequency. It maintains a two-entry sliding window history; during warm-up, the single available snapshot is duplicated to ensure the input tensor is always fully populated. The primary decision path calls `CNNController::predictFrequencyIndex()`, which returns a class index in $\{0, 1, 2\}$ mapped to 1.0, 1.5, and 2.0 GHz via `CNN_CLASS_TO_FREQ_MHZ`. A frequency transition is applied only when the predicted frequency differs from the current one. If the ONNX model is unavailable, the decision is redirected to the rule-based fallback.

`CNNController` loads the ONNX model at simulation startup via the ONNX Runtime C++ API and runs single-threaded inference to avoid non-deterministic interference with the gem5 event loop. On each decision, the two history snapshots are assembled into a 28-element input

vector, normalised using per-feature mean and standard deviation compiled into `cnn_scaler_params.hh`, and passed to the session. The three output logits are reduced to a class index via `argmax` and returned to `FlowRegulator`. If the model cannot be loaded or inference fails, the decision falls back to the rule-based policy without aborting the simulation.

4.2.4 Training Data Collection

Three PARSEC benchmarks are used: blackscholes, canneal, and ferret. Blackscholes represents a compute-intensive workload with regular low-volume network traffic, while canneal and ferret exhibit more irregular and communication-intensive access patterns. Canneal serves as both the training data source and a primary evaluation workload; blackscholes and ferret provide additional evaluation settings.

Each trajectory is driven by a frequency schedule supplied via `-freq-schedule`. For trajectory index n, the schedule consists of $n + 1$ windows at 1.0 GHz followed by a single window at the target frequency, giving a total duration of $(n + 2) \times 20{,}000{,}000$ ticks. This design ensures all trajectories share an identical prefix, so the final window's observations reflect only the effect of the target frequency. To limit logging overhead, `-csv-tail-windows=3` restricts output to the last three windows: the two preceding 1.0 GHz windows and the switched-frequency window, corresponding directly to `hist0`, `hist1`, and the label generation row.

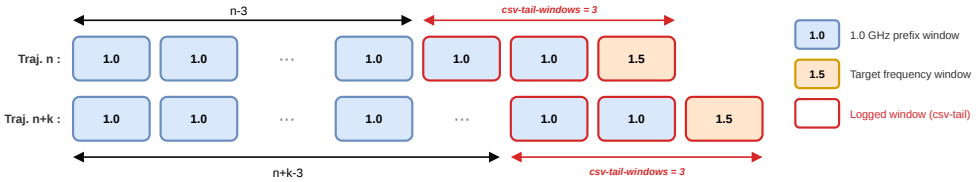


Figure 4.3: Trajectory structure for training data collection.

4.2.5 Rule-Based Controller Implementation

The rule-based controller is implemented as a stateless function evaluated once per monitoring window. Given the observation vectors `hist0` and `hist1` produced by the `FlowObserver`, the controller emits one of three frequency levels $\{1.0, 1.5, 2.0\}$ GHz. Six scalar features are extracted from the two observation vectors: from `hist1` (window t), total packet latency L_t^{tot},

network latency L_t^{net}, average link utilisation U_t, and injection rate I_t; from `hist0` (window $t-1$), total packet latency L_{t-1}^{tot} and network latency L_{t-1}^{net}. The rule is implemented as a `C++` function in the `FlowRegulator` module of Garnet, replacing the MLP inference call in the closed-loop simulation pipeline. The same `FlowObserver` instrumentation feeds both controllers, ensuring that any performance differences observed in the evaluation arise solely from the decision logic rather than from differences in the input signals.

4.2.5.1 Threshold Values

Threshold values were set by inspecting the feature distributions recorded in the full-system PARSEC runs, since the latency and load of the controller must respond to real application traffic rather than synthetic load. Values were rounded to engineering-friendly magnitudes to reflect the way such rules are typically constructed in practice. The resulting parameters are listed in Table 4.2.

Table 4.2: Threshold parameters of the rule-based baseline.

Indicator	Parameter	Value
High latency	$\theta_{\text{tot}}^{\text{hi}}$	50000 cycles
	$\theta_{\text{net}}^{\text{hi}}$	15000 cycles
Mid latency	$\theta_{\text{tot}}^{\text{mid}}$	24000 cycles
	$\theta_{\text{net}}^{\text{mid}}$	6000 cycles
Rising trend	ρ_{tot}	1.10
	ρ_{net}	1.40
High load	θ_I^{hi}	0.30
	θ_U^{hi}	0.025
Mid load	θ_I^{mid}	0.04
	θ_U^{mid}	0.005

4.2.5.2 Algorithmic Structure

The congestion indicators are realized as the following boolean predicates:

$$\texttt{HighLat} \equiv (L_t^{\text{tot}} > \theta_{\text{tot}}^{\text{hi}}) \vee (L_t^{\text{net}} > \theta_{\text{net}}^{\text{hi}}) \tag{4.3}$$

$$\texttt{MidLat} \equiv (L_t^{\text{tot}} > \theta_{\text{tot}}^{\text{mid}}) \vee (L_t^{\text{net}} > \theta_{\text{net}}^{\text{mid}}) \tag{4.4}$$

$$\texttt{Rising} \equiv (L_t^{\text{tot}} > \rho_{\text{tot}} \cdot L_{t-1}^{\text{tot}}) \vee (L_t^{\text{net}} > \rho_{\text{net}} \cdot L_{t-1}^{\text{net}}) \tag{4.5}$$

$$\texttt{HighLoad} \equiv (I_t > \theta_I^{\text{hi}}) \wedge (U_t > \theta_U^{\text{hi}}) \tag{4.6}$$

$$\texttt{MidLoad} \equiv (I_t > \theta_I^{\text{mid}}) \vee (U_t > \theta_U^{\text{mid}}) \tag{4.7}$$

These predicates are combined into a three-tier priority scheme, as shown in Algorithm 1.

Algorithm 1 Rule-based frequency selection

Require: `hist0` (window $t-1$), `hist1` (window t)
Ensure: Target frequency $f \in \{1.0, 1.5, 2.0\}$ GHz

```
if HighLat ∨ HighLoad then
    return 2.0
else if MidLat ∨ (Rising ∧ MidLoad) then
    return 1.5
else
    return 1.0
end if
```

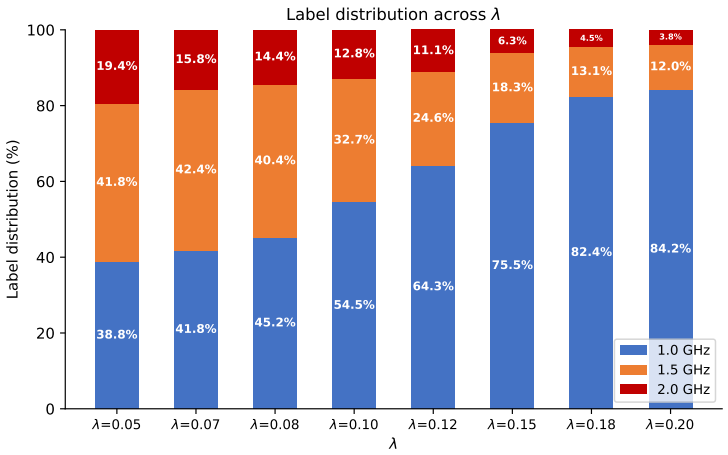
4.3 Offline Training Pipeline

4.3.1 Label Generation and λ Sweep

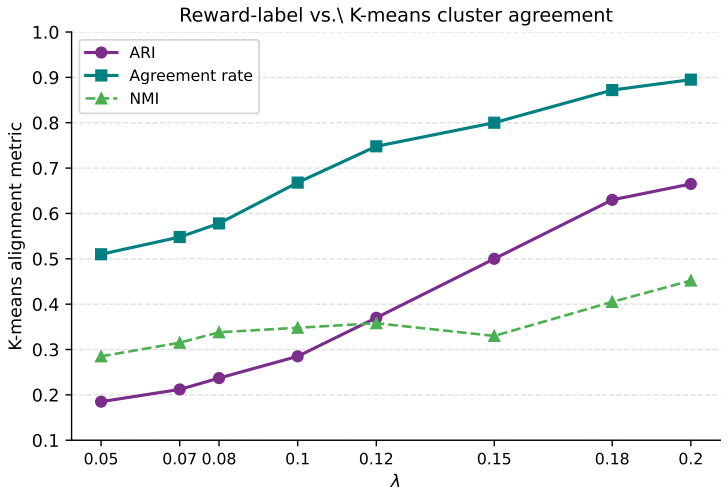
Label generation follows the reward-based procedure described in Section 3.5.1. Rather than fixing λ to a single value, a sweep over eight candidates is conducted to understand how the trade-off coefficient affects label distribution and downstream classification performance, and to select the three λ values used in the final controller variants.

For each candidate $\lambda \in \{0.05, 0.07, 0.08, 0.10, 0.12, 0.15, 0.18, 0.20\}$, the full pipeline from label generation through model training and cross-validation is executed independently. Figure 4.4 summarises the results.

Figure 4.4a shows that increasing λ progressively shifts the label distribution: at $\lambda = 0.05$ the three classes are roughly equally represented



(a) Effect of λ on label distribution across the three frequency classes.



(b) Agreement between reward-based labels and K-means clusters as a function of λ.

Figure 4.4: Effect of λ on label distribution and cluster agreement.

(38.8 % / 41.8 % / 19.4 %), whereas at $\lambda = 0.18$ the 1.0 GHz label dominates at 82.4 %. Figure 4.4b shows that agreement between reward-based labels and K-means clusters increases monotonically with λ, confirming that higher λ values produce more power-oriented labels that are geometrically separable in feature space.

Accordingly, three values are selected to represent qualitatively distinct operating priorities: $\lambda = 0.05$ (HighPerf-CNN), $\lambda = 0.10$ (Balanced-CNN),

and $\lambda = 0.18$ (LowPower-CNN). Table 4.3 summarizes the resulting label distributions and classification performance. The latency guard threshold $\tau_{\text{lat}} = 1.03$ is held fixed across all runs; a sensitivity check confirms that varying it between 1.01 and 1.05 affects fewer than 3 % of samples.

Table 4.3: Selected λ values, label distributions, and cross-validation performance. Label percentages are computed over the full labelled dataset of 1,117 samples. Balanced accuracy (BalAcc) is the mean over three folds; standard deviation is shown in parentheses.

Strategy	λ	**1.0 GHz**	**1.5 GHz**	**2.0 GHz**	**BalAcc**
HighPerf-CNN	0.05	38.8%	41.8%	19.4%	0.794 (0.011)
Balanced-CNN	0.10	54.5%	32.7%	12.8%	0.766 (0.025)
LowPower-CNN	0.18	82.4%	13.1%	4.6%	0.778 (0.033)

The monotonic increase in K-means agreement with λ does not imply that the three frequency classes become increasingly separable in the feature space. Figure 4.5 presents a K-means analysis ($K = 3$) at $\lambda = 0.10$ via PCA projection. The first principal component retains 79.2 % of the total variance, indicating a dominant structure aligned with network load intensity.

Panel (a) shows the reward-based label assignments distributed along PC1. Panel (b) reveals that K-means assigns 941 of 1117 samples to the 1.0 GHz cluster, while only 82 samples form a distinct 1.5 GHz cluster—versus 365 reward-labelled 1.5 GHz samples. Panel (c) quantifies this: 316 of those 365 samples are absorbed into the 1.0 GHz cluster (agreement rate 66.3 %, ARI = 0.286), indicating that the 1.5 GHz boundary corresponds to a transitional region with substantial class overlap rather than a geometrically distinct state. By contrast, 92 of 143 reward-labelled 2.0 GHz samples are correctly recovered, confirming that high-traffic states are more distinct. Panel (d) shows that disagreements are concentrated in the low-to-mid PCA region, while the 2.0 GHz region exhibits comparatively fewer conflicts.

This structural ambiguity in the 1.5 GHz boundary is an inherent property of the feature space and directly motivates the use of temporal context in the CNN architecture.

4.3.2 Model Selection

Three separate models are trained, one for each selected λ value. Each model is trained independently with its own label set, its own `StandardScaler` fitted on the corresponding training split, and its own ONNX export. The three

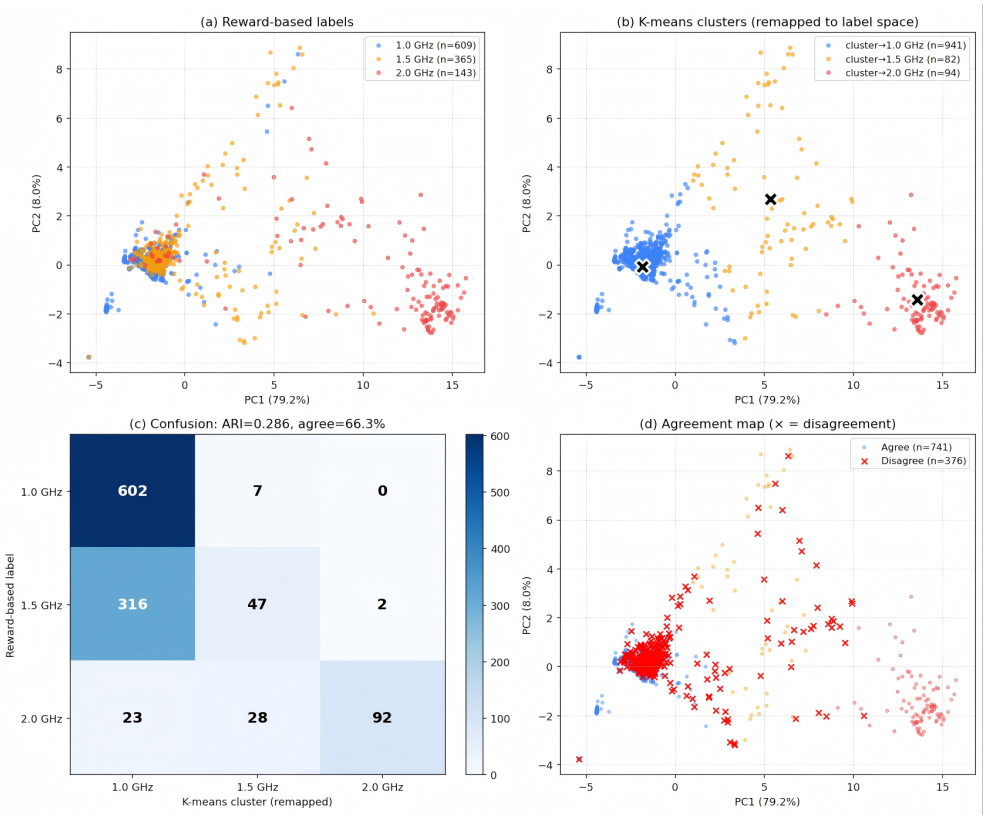


Figure 4.5: K-means analysis ($K = 3$, $\lambda = 0.10$, $N = 1117$) on standardised features via PCA. (a) Reward-based labels; (b) K-means clusters remapped to frequency labels; (c) Confusion matrix (ARI = 0.286, agreement = 66.3 %); (d) Spatial agreement map.

models share the same CNN architecture described in Section 3.5.3; the only difference between them is the label assignment derived from the respective λ.

Within each training run, the fold with the highest balanced validation accuracy is selected as the final model. Balanced accuracy is preferred over raw accuracy as the selection criterion because the label distribution is skewed towards the 1.0 GHz class under larger λ values; a model that predicts the majority class for every input would achieve high raw accuracy while being entirely useless in practice. Balanced accuracy treats each class recall equally and is therefore a more meaningful indicator of whether the model has learned to distinguish all three frequency levels.

Each selected checkpoint is exported to ONNX and paired with a dedicated `cnn_scaler_params.hh` header generated from its own scaler. The

three model files and headers are stored in separate subdirectories to prevent accidental cross-contamination between strategies. At simulation time, the desired strategy is selected by pointing `-cnn-model-path` to the corresponding ONNX file; no recompilation is required to switch between strategies.

4.3.3 Ablation Study

The ablation experiments described in Section 3.5.4 are executed using the same labeled dataset and cross-validation protocol as the main training runs. All configurations share the CNN architecture and hyperparameters from Section 3.5.3; only the input feature set or history window size is varied.

4.3.3.1 Feature Group Ablation

For each ablation configuration, the corresponding columns are dropped from `labeled_data.csv` before constructing the feature matrix. The remaining features are flattened across both history windows in the same order as the baseline, so the only structural change to the model is a reduced `input_dim` and `num_features`. A `StandardScaler` is fitted independently on each training fold to ensure that the reduced feature set is normalised without leakage. Per-class weights are recomputed from the training fold of each configuration, so class-imbalance handling is consistent across all runs. The baseline uses all 14 features (input_dim $= 28$); each ablated variant reduces this by the size of the removed group.

4.3.3.2 History Window Ablation

The $H = 1$ variant retains only the snapshot immediately preceding the decision point: only the `hist1_*` columns are selected from `labeled_data.csv`, as `hist1` is the more recent of the two recorded windows. With a single window the two-step convolution no longer applies, so this variant uses a flat fully-connected network over the 14 input features. The comparison measures the overall contribution of the two-window design, including both the extra window and the convolution over it, while holding the feature set, label set, and training protocol fixed.

All ablation runs are performed at $\lambda = 0.05$, using the HighPerf-CNN label set. Results are reported as mean balanced accuracy over three folds together with the drop relative to the full-feature, $H = 2$ baseline.

4.4 Evaluation Setup

Two experimental conditions are compared: a fixed-frequency baseline running continuously at 1.0 GHz, and a DVFS-enabled simulation in which the `FlowRegulator` issues frequency decisions at every observation window boundary. Three DVFS-enabled conditions are evaluated, one per strategy (HighPerf-CNN, Balanced-CNN, and LowPower-CNN), each using the model selected from the best-performing cross-validation fold as described in Section 4.3.2.

The PARSEC benchmarks BlackScholes, Canneal, and Ferret are selected as evaluation workloads. Both benchmarks are executed within the ROI, which is bounded by the `m5_work_begin` and `m5_work_end` magic instructions inserted into the benchmark source. The ROI tick is identified from terminal output logging during an initial profiling run and used to set a consistent simulation start point across all experimental conditions. All simulations within a comparison pair are terminated at the same absolute tick value via `-abs-max-tick`, ensuring that performance metrics are accumulated over identical simulation intervals and that differences in throughput and latency are attributable to the frequency control policy rather than to differences in simulation duration.

Chapter 5

Results and Analysis

5.1 CNN Controller Training Results

Three CNN controllers were trained, each corresponding to a distinct frequency-selection policy determined by the reward trade-off parameter $\lambda \in \{0.05, 0.10, 0.18\}$. This section presents their training and evaluation results.

5.1.1 Cross-Validation Performance

Each model was evaluated via 3-fold cross-validation across three folds. Two accuracy metrics are reported: raw accuracy measures the overall fraction of correctly classified samples, while balanced accuracy computes the mean per-class recall, giving equal weight to each frequency class regardless of its prevalence. The distinction matters particularly for the power-oriented model, where the dominance of the 1.0 GHz label means a controller could achieve high raw accuracy simply by predicting the majority class, while balanced accuracy exposes the true per-class performance. Table 5.1 summarises the cross-validation outcomes.

Table 5.1: 3-fold cross-validation results for the three CNN controllers.

Model	λ	**Bal. Acc. (mean $\pm$ std)**	**Raw Acc. (mean $\pm$ std)**
HighPerf-CNN	0.05	0.7936 ± 0.0112	0.7968 ± 0.0065
Balanced-CNN	0.10	0.7664 ± 0.0250	0.7296 ± 0.0117
LowPower-CNN	0.18	0.7782 ± 0.0330	0.9051 ± 0.0147

All three models achieve a mean balanced accuracy well above the random-guess baseline of 0.333, confirming that the frequency-selection policies

encoded in each CNN controller are learnable from the simulation features.

The HighPerf-CNN model ($\lambda = 0.05$) yields the highest and most stable balanced accuracy (0.794 ± 0.011), with raw accuracy closely aligned at 0.797, reflecting the relatively uniform label distribution across the three frequency classes.

The Balanced-CNN model ($\lambda = 0.10$) achieves a balanced accuracy of 0.766 ± 0.025, with notably higher variance across folds than the HighPerf-CNN model, suggesting sensitivity to fold composition.

The LowPower-CNN model ($\lambda = 0.18$) achieves a raw accuracy of 0.905, substantially higher than its balanced accuracy of 0.778 ± 0.033, reflecting the dominance of the 1.0 GHz class in the training data; the higher variance indicates sensitivity to fold composition due to the scarcity of 1.5 and 2.0 GHz training examples.

5.1.2 Training Dynamics

Figures 5.1–5.3 present the training loss and validation balanced accuracy curves for a representative fold of each model. All three controllers converge within the first 100 epochs, with training loss decreasing steadily throughout.

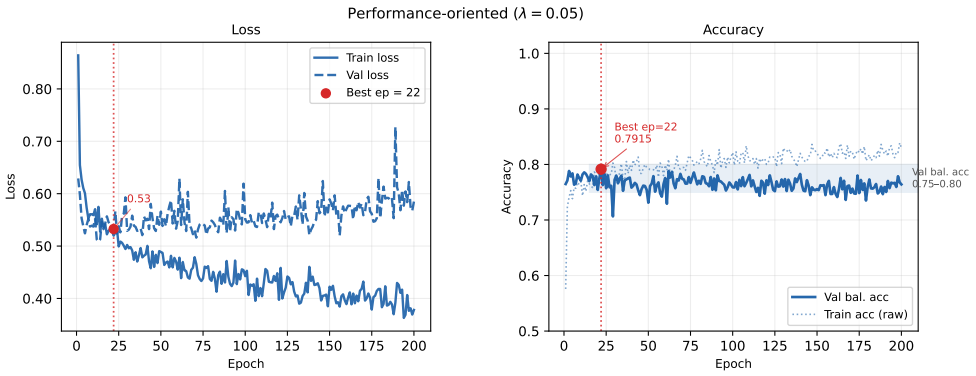


Figure 5.1: Training loss and validation balanced accuracy curves for the HighPerf-CNN model.

The HighPerf-CNN model ($\lambda = 0.05$) reaches its best checkpoint at epoch 22, after which training loss continues to decline to approximately 0.37 while validation balanced accuracy oscillates in the range 0.75–0.80, indicating overfitting in later epochs.

The Balanced-CNN model ($\lambda = 0.10$) reaches its best checkpoint at epoch 33, after which validation loss rises and fluctuates considerably. The notably early best epoch of fold 1 (epoch 2) reflects sensitivity to fold

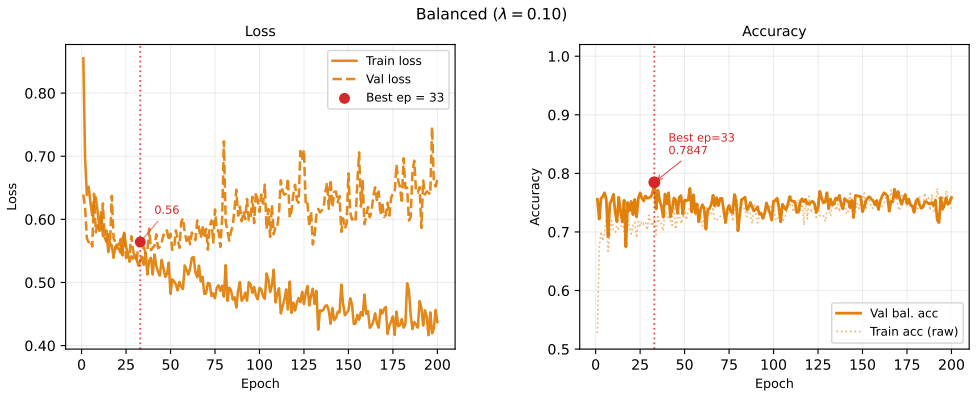


Figure 5.2: Training loss and validation balanced accuracy curves for the Balanced-CNN model.

composition, consistent with the higher standard deviation of 0.025 observed in cross-validation, and attributable to the less uniform label distribution at this λ.

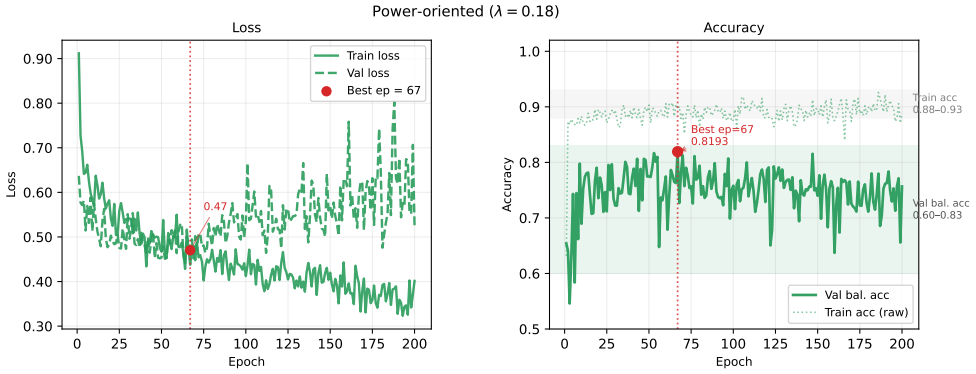


Figure 5.3: Training loss and validation balanced accuracy curves for the LowPower-CNN model.

The LowPower-CNN model ($\lambda = 0.18$) reaches its best checkpoint at epoch 67, but exhibits the most volatile validation balanced accuracy across all 200 epochs, ranging from 0.60 to 0.83. The persistent gap between training raw accuracy (∼0.88–0.93) and validation balanced accuracy reflects the severe class imbalance in the training data rather than genuine overfitting.

In all three cases, the deployed model corresponds to the best checkpoint rather than the final epoch, ensuring that transient degradation in later epochs does not affect inference performance.

5.1.3 Confusion Matrix Analysis

Figure 5.4 presents the normalized confusion matrices for each model. A consistent pattern across all three CNN controllers is that dominant misclassification occurs at adjacent frequency boundaries: 1.0 GHz samples are occasionally predicted as 1.5 GHz, and 1.5 GHz samples are split between the two neighbouring classes. Direct confusion between 1.0 GHz and 2.0 GHz is negligible in all three models, confirming that the CNN controllers have learned a meaningful ordinal structure in the frequency space.

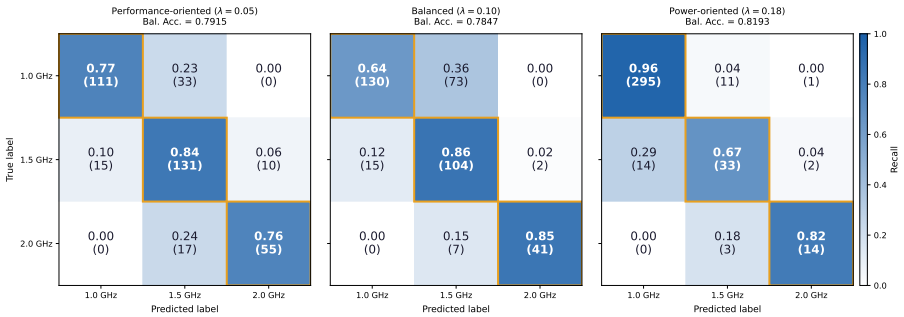


Figure 5.4: Confusion matrices of each CNN controller.

The HighPerf-CNN model ($\lambda = 0.05$) exhibits a relatively balanced recall distribution, with per-class recalls of 77.1 %, 84.5 %, and 76.4 % for 1.0, 1.5, and 2.0 GHz respectively. The 1.0 GHz class is the hardest to predict, with 33 out of 144 samples misclassified as 1.5 GHz, suggesting that under low λ the reward margin between the two lowest frequencies is narrow and the boundary between them is less clearly defined.

The Balanced-CNN model ($\lambda = 0.10$) shows a pronounced asymmetry: the 1.0 GHz recall drops to 64.0 % while 1.5 and 2.0 GHz recalls reach 86.0 % and 85.4 %. The majority of misclassified 1.0 GHz samples are predicted as 1.5 GHz (73 out of 203), indicating that at this λ the controller systematically errs towards scaling up when the traffic state is ambiguous.

The LowPower-CNN model ($\lambda = 0.18$) achieves the highest recall on the dominant 1.0 GHz class (96.1 %) but struggles with the minority 1.5 GHz class (67.3 %), which has only $\sim$49 validation samples per fold. The 2.0 GHz recall of 82.4 % is robust given its scarcity, likely because the few 2.0 GHz labels at high λ correspond to unambiguously high-traffic conditions that are straightforward to identify from the input features.

The misclassification patterns across all three models are consistent with the ordinal nature of the frequency classes, where errors predominantly occur between neighboring frequency levels rather than across distant ones.

Taken together, these results confirm that the reward-based labelling scheme successfully encodes distinct frequency-selection policies into the three CNN controllers.

All three models are therefore retained for downstream DVFS control evaluation, where their differing frequency-selection biases are expected to produce measurable differences in latency and power behavior under varied traffic conditions.

5.1.4 Ablation Study

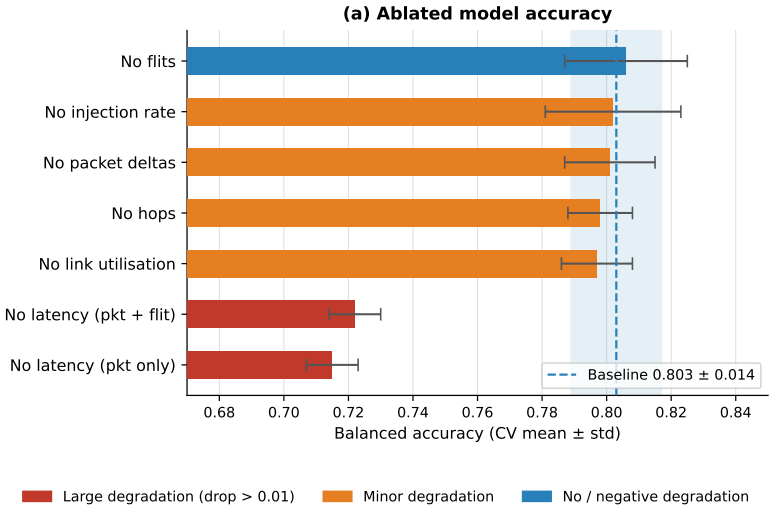
To assess the contribution of individual feature groups and the history window design, two ablation experiments were conducted on the HighPerf-CNN model ($\lambda = 0.05$) using the same 3-fold cross-validation protocol. The baseline balanced accuracy reported here (0.803 ± 0.014) reflects a separate training run and may differ slightly from the cross-validation results in Table 5.1 due to random initialisation.

5.1.4.1 Feature Group Ablation

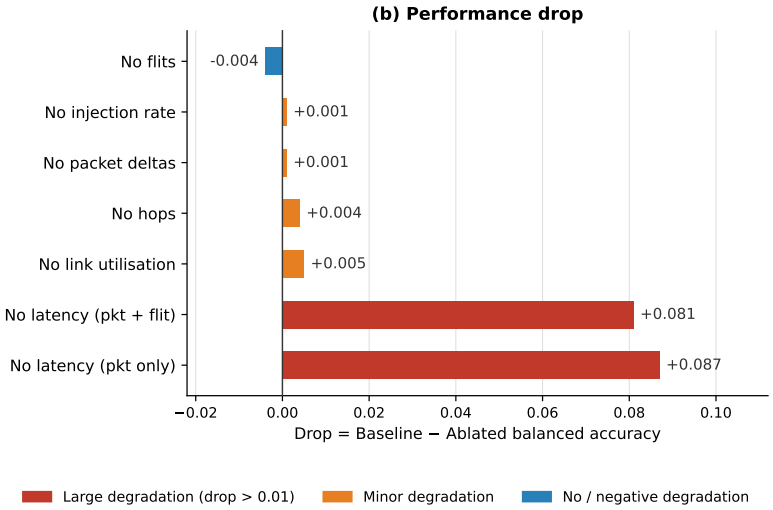
Figures 5.5 reports the effect of removing each semantic feature group from the full 14-feature input set. Removing all latency-related features—both packet-level (`AvgTotalLatency_pkt`, `AvgNetworkLatency_pkt`, `AvgQueueingLatency_pkt`) and flit-level equivalents—produces the largest drop in balanced accuracy (0.081), confirming that latency is the dominant signal for frequency selection. This result is consistent with latency being the most direct observable indicator of network congestion: removing only the three packet-level latency features produces a drop of 0.087, compared to just 0.004 for removing the three flit-throughput features, indicating that the performance gap is driven by information quality rather than feature count.

The marginally larger drop from removing packet-level latency alone compared to removing both packet- and flit-level latency together (0.087 vs. 0.081) is consistent with the slight redundancy of flit-level latency features.

All remaining feature groups (`InjectionRate`, `AvgHops`, `AvgLinkUtil`, packet deltas) produce drops no larger than 0.005, indicating that the non-latency features are largely redundant given the latency signal, and that the 14-feature input set contains no single catastrophic dependency beyond latency.



(a) Ablated model balanced accuracy.



(b) Performance drop relative to baseline.

Figure 5.5: Feature group ablation results.

5.1.4.2 History Window Ablation

Table 5.6 compares the full two-window CNN+MLP model against a single-window baseline that retains only the most recent observation (`hist1`) using a flat MLP architecture. Reducing the history window from two to one lowers balanced accuracy by 0.041 (0.805 → 0.764), a moderate but consistent degradation that exceeds the noise level observed across folds.

The source of this improvement can be attributed to two complementary mechanisms. For absolute-valued features such as latency, the second window supplies first-order trend information: the model can implicitly compute the rate of change in congestion from the difference between `hist0` and `hist1`, which is not observable from a single snapshot. For delta-valued features such as `PacketsReceivedDelta` and `FlitsReceivedDelta`, which already encode one-step increments, the second window provides second-order information about how throughput is accelerating or decelerating. Given that the feature ablation identified latency as the dominant input group, the window benefit is most likely driven by the latency trend component rather than the throughput second-order signal.

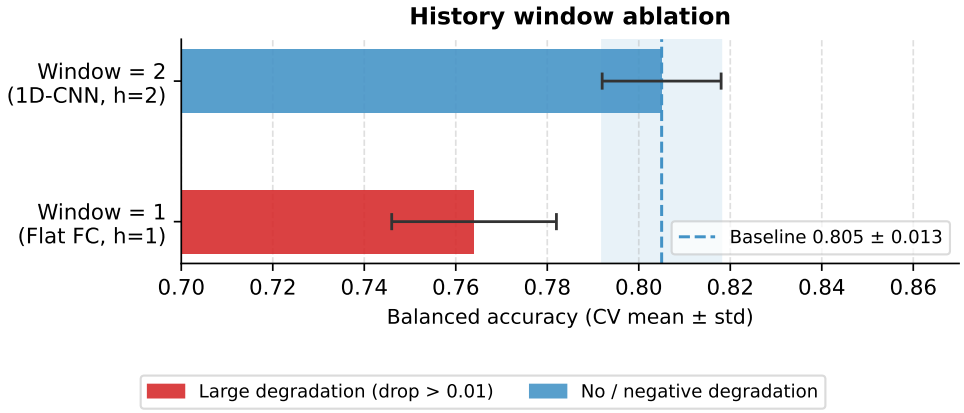


Figure 5.6: History window ablation results ($\lambda = 0.05$, 3-fold CV).

Taken together, the ablation results justify both the feature selection and the two-window design adopted in this work. Latency features are indispensable, the history window provides meaningful trend information at moderate cost, and the remaining features contribute negligibly—suggesting that a more compact feature set centred on latency and a two-step history would preserve most of the model's predictive capacity.

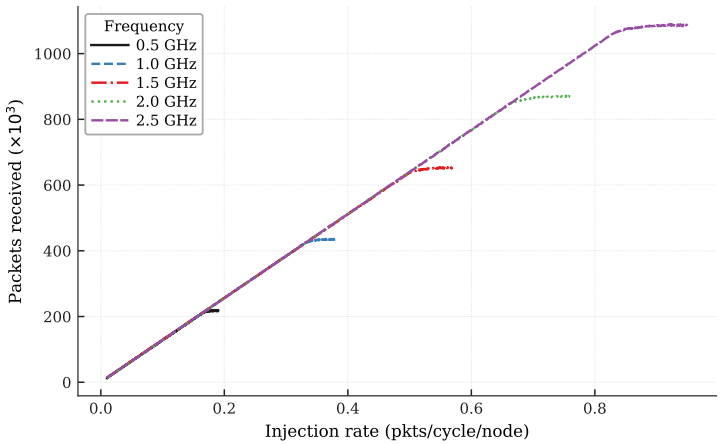
5.2 NoC DVFS Evaluation

5.2.1 NoC Performance Characterisation

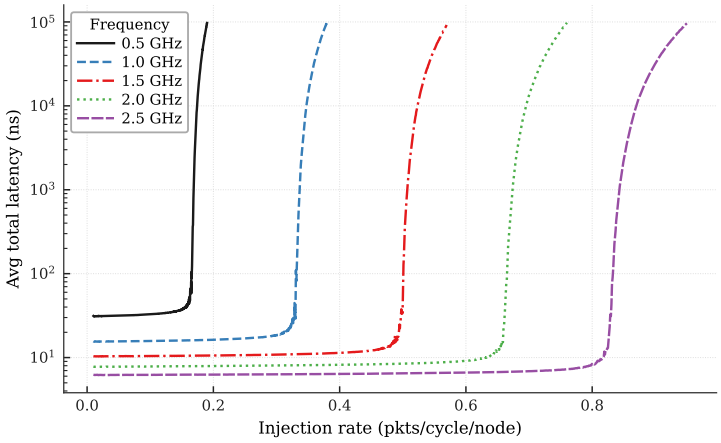
To motivate the adaptive frequency control approach, a characterization study was conducted using Garnet Standalone with uniform random synthetic traffic across five fixed frequencies (0.5–2.5 GHz). For each frequency, sweeps were

terminated upon saturation, so the injection rate range varies accordingly, ranging from 0.01 to 0.95 pkts/cycle/node.

Figure 5.7a shows the throughput curves across five frequency configurations. Each configuration saturates at a distinct injection rate; beyond saturation, throughput plateaus regardless of offered load. The saturation point shifts to higher injection rates with increasing frequency, and peak throughput scales approximately linearly with frequency. Below saturation, however, all configurations achieve equal throughput for a given injection rate, meaning that lower frequencies impose no capacity penalty under light traffic.



(a) Throughput



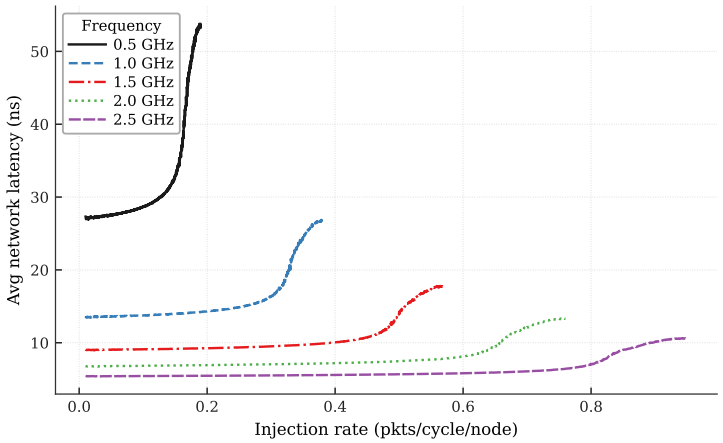
(b) Total Packet Latency

Figure 5.7: Throughput and total packet latency under five fixed frequencies.

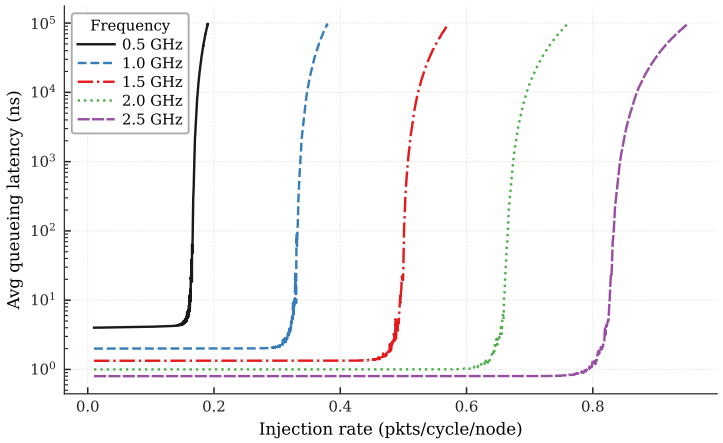
This difference motivates adaptive frequency control: the operating

frequency should follow traffic load, running lower to save power under light traffic and higher to maintain capacity under heavy traffic.

Total packet latency in Figure 5.7 is decomposed into network and queueing components in Figure 5.8.



(a) Network Latency



(b) Queueing Latency

Figure 5.8: Network latency and queueing latency under five fixed frequencies.

Network latency in Figure 5.8a differs across frequencies in absolute time, reflecting longer per-hop traversal at lower clock speeds. This difference is purely a function of frequency: when normalized to cycles at the simulated frequency, the five curves collapse to a single trace, confirming that routing and link traversal costs depend only on topology. Queueing latency in Figure 5.8b behaves differently. At low injection rates it remains small and similar across all frequencies, but rises sharply as each configuration

approaches its saturation point, with lower frequencies reaching this onset earlier.

The spatial distribution of traffic load across the 8×8 mesh is shown in Figure 5.9. Buffer read counts averaged over the observation window reveal a pronounced centre-heavy pattern, with routers near the mesh centre (R27, R28, R35, R36) sustaining approximately three times the load of corner routers under XY routing.

This non-uniform distribution means that a single global frequency setting is an inherently imprecise instrument: corner routers are over-provisioned while centre routers risk congestion. It also provides spatial context for interpreting the queueing latency behaviour observed above, as the sharp latency increase near saturation originates disproportionately from these central hotspot routers.

These observations inform two requirements for the DVFS controller evaluated in the remainder of this chapter: the ability to reduce frequency under light load without performance loss, and the ability to detect congestion onset before queueing latency degrades substantially.

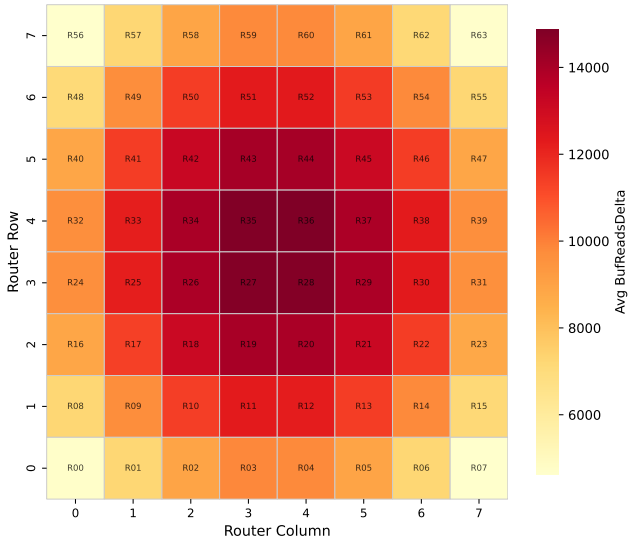


Figure 5.9: Per-router buffer read activity in an 8×8 mesh NoC under synthetic traffic (1.0 GHz, injection rate 0.084).

Since the controller operates at the network level rather than per-router, these spatial differences are not directly actionable, and per-router frequency scaling remains a direction for future work.

5.2.2 Fixed-Frequency Benchmark Evaluation

Figure 5.10 shows the execution time of each benchmark at 1.5 GHz and 2.0 GHz relative to the 1.0 GHz baseline. For Canneal and Ferret the simsmall input set is used, while Blackscholes uses the simlarge input set.

The three benchmarks respond very differently to frequency scaling. Canneal sees the largest reduction, dropping to 84.74% of the aseline at 1.5 GHz and 82.39% at 2.0 GHz. Blackscholes improves only modestly, reaching 97.03% at 1.5 GHz and 96.91% at 2.0 GHz, with little additional gain beyond 1.5 GHz. Ferret is essentially unaffected: execution time stays at 100.02% and 99.57% across the two higher frequencies.

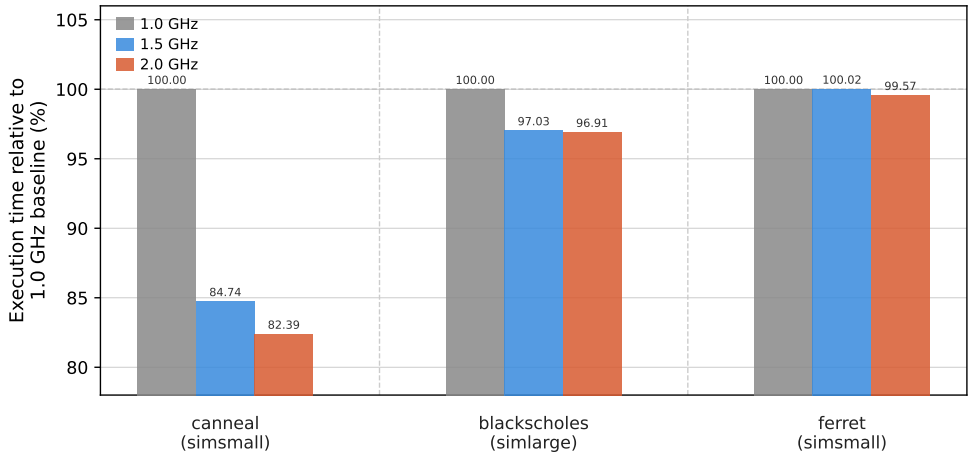


Figure 5.10: Execution time relative to the 1.0 GHz baseline for each benchmark across fixed-frequency configurations.

Network latency drops substantially as frequency increases: from 13.7 ns at 1.0 GHz to 4.7 ns at 2.0 GHz for canneal, from 14.7 ns to 3.8 ns for blackscholes, and from 8.6 ns to 3.3 ns for ferret. However, DRAM latency barely moves. It stays around 37 ns for Canneal, 31–34 ns for Blackscholes, and 35 ns for ferret, and remains well above the network latency at every frequency setting. Because DRAM latency does not change with NoC frequency, it sets a floor on execution time that frequency scaling cannot overcome. For Ferret, this explains why a 61% drop in network latency produces no measurable speedup: the workload spends most of its time waiting on memory, not on the network.

Canneal and Blackscholes both have similar network latency profiles at 1.0 GHz, yet canneal benefits far more from frequency scaling. The data available here does not fully account for this difference; a more detailed

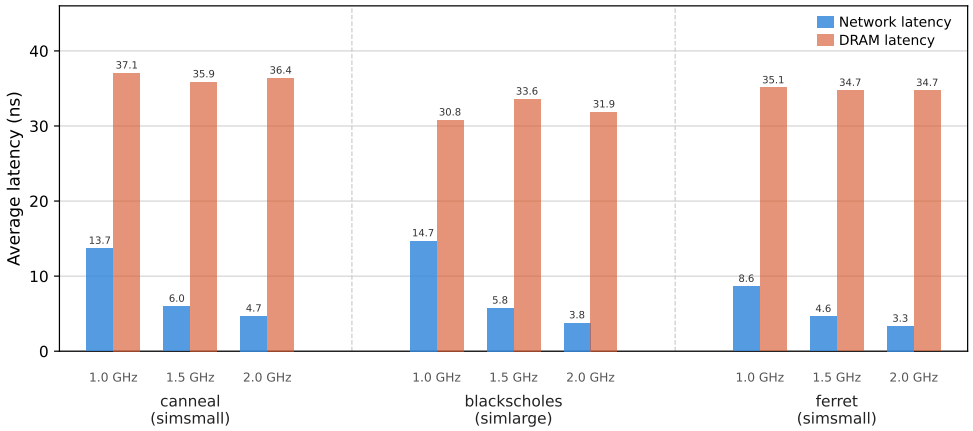


Figure 5.11: Average packet network latency and DRAM memory access latency across fixed-frequency configurations.

breakdown of memory access patterns per workload would be needed to explain it. What the results do show is that the value of running at a higher frequency depends heavily on the workload: unnecessary for Ferret, marginally useful for BlackScholes, and clearly worthwhile for Canneal.

5.2.3 Controller Performance

Figure 5.12 shows the total execution time of each controller as a percentage of the 1.0 GHz fixed-frequency baseline ($t_{\text{base}} = 45{,}411{,}600{,}100$ ticks), evaluated on the canneal simsmall benchmark. A lower value indicates shorter execution time relative to the baseline.

As shown in Figure 5.12, the fixed 2.0 GHz baseline gives the shortest execution time, completing the benchmark in 82.4% of the 1.0 GHz baseline time. The fixed 1.5 GHz configuration also improves execution time to 84.7%. Among the adaptive controllers, HighPerf-CNN-DVFSController achieves the shortest execution time, 86.5%, slightly outperforming the rule-based controller at 87.4%. Balanced-CNN-DVFSController obtains a similar result at 87.0%, while LowPower-CNN-DVFSController is more conservative and completes the benchmark in 91.9% of the baseline time.

The difference between the rule-based controller and HighPerf-CNN-DVFSController is small, at under one percentage point, but HighPerf-CNN-DVFSController achieves shorter execution times. The larger gap observed in LowPower-CNN-DVFSController is expected, given that its reward function penalizes high-frequency selections more heavily, leading to

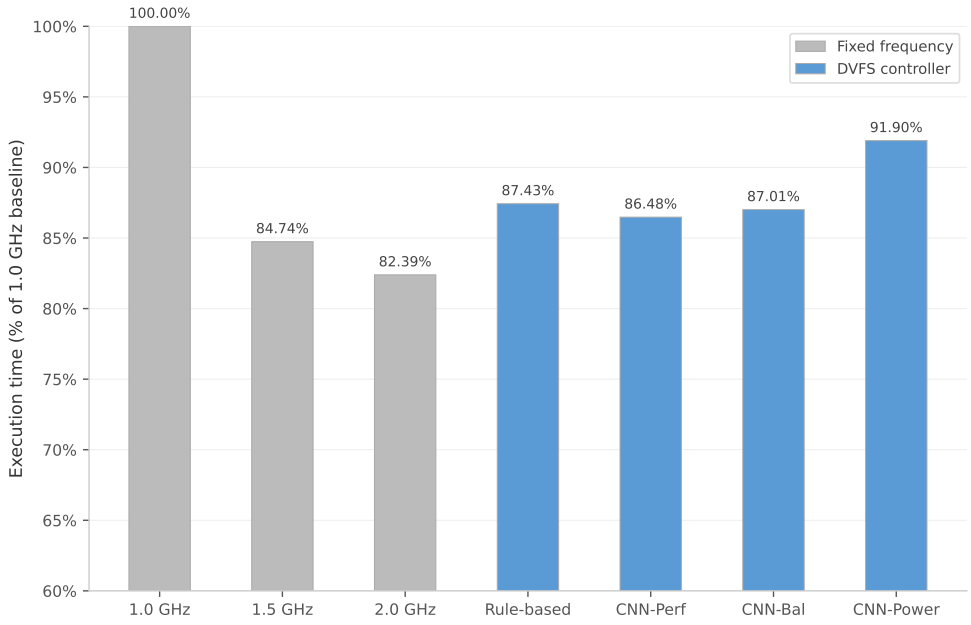


Figure 5.12: Execution time improvement of DVFS control strategies

more conservative frequency decisions and longer execution times.

5.3 Summary

This chapter has presented results across offline classification performance of three CNN controllers and closed-loop DVFS control performance under full-system simulation.

On the offline axis, all three controllers achieve mean balanced accuracy well above the random-guess baseline, confirming that the frequency-selection policies encoded by the reward function are learnable from the collected simulation features. The HighPerf-CNN model ($\lambda = 0.05$) achieves the highest and most stable balanced accuracy (0.794 ± 0.011), while the Balanced-CNN model ($\lambda = 0.10$, 0.766 ± 0.025) and LowPower-CNN model ($\lambda = 0.18$, 0.778 ± 0.033) models exhibit greater sensitivity to fold composition, attributable to the increasingly skewed label distributions at higher λ.

Across all three models, misclassification is concentrated at adjacent frequency boundaries. The ablation study suggests that latency features are the dominant input signal. The two-window CNN configuration also outperforms the single-window baseline, reflecting both the additional

historical information and the difference in model architecture.

On the closed-loop axis, the fixed-frequency characterization reveals that the benefit of frequency scaling is strongly workload-dependent. canneal reduces execution time to 84.74 % and 82.39 % of the 1.0 GHz baseline at 1.5 and 2.0 GHz, while Ferret remains essentially unchanged across all fixed-frequency settings. This divergence shows that a reduction in network latency does not always translate into a proportional improvement in execution time, and that the benefit of frequency scaling varies substantially across workloads.

Under adaptive control on canneal, HighPerf-CNN-DVFSController achieves the shortest execution time at 86.48 % of the 1.0 GHz baseline, outperforming the rule-based controller at 87.43 %. Balanced-CNN-DVFSController reaches 87.01 %, and LowPower-CNN-DVFSController, operating under the most conservative reward weighting, reaches 91.90 %.

The ordering of all four controllers is consistent with their respective frequency-selection biases: a higher penalty on power costs leads to more conservative frequency decisions and, correspondingly, longer execution times. Taken together, these results confirm that the reward-parameterized training procedure produces controllers with meaningfully differentiated behaviors, and that the HighPerf-CNN can match and exceed a carefully rule-based controller without requiring manual threshold calibration.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This thesis investigated whether a supervised learning approach can replace hand-crafted rule-based logic for DVFS control in Network-on-Chip systems, and whether the resulting controller can achieve a better mechanism for adjusting the performance–power trade-off than static frequency assignment. Three 1D CNN controllers were designed, trained on data collected from gem5 simulation, and deployed within the same simulation environment via ONNX Runtime for closed-loop evaluation. Each controller corresponds to a distinct operating priority (HighPerf-CNN, Balanced-CNN, and LowPower-CNN) defined by the reward trade-off parameter λ.

Offline evaluation confirms that all three policies are learnable from the collected NoC features. The HighPerf-CNN model achieves a mean balanced accuracy of 0.794 ± 0.011 across three cross-validation folds, with misclassification concentrated at the 1.0–1.5 GHz boundary, an overlap in the feature space identified by K-means analysis rather than an artifact of the model architecture. The ablation study establishes that latency features account for the majority of the model's classification performance, and that the two-window temporal design provides a consistent improvement of 0.041 in balanced accuracy over a single-window baseline by capturing congestion trend information across observation windows. These findings support the decisions made in the feature engineering and architecture stages.

In closed-loop evaluation on the canneal benchmark, HighPerf-CNN-DVFSController achieves an execution time of 86.48 % relative to the 1.0 GHz fixed-frequency baseline, outperforming the rule-based controller at 87.43 %. Balanced-CNN-DVFSController reaches 87.01 % and LowPower-

CNN-DVFSController reaches 91.90 %, with the ordering across all four controllers consistent with each controller's tendency to select higher frequencies. This confirms that the learned policy can operate effectively in a full-system simulation setting.

The fixed-frequency characterization further reveals that the value of adaptive control is strongly dependent on workload. Ferret shows negligible execution time differences across all control strategies, which underscores the importance of evaluating DVFS controllers across benchmarks with varying communication intensities.

The main result shows that a CNN-based controller can be integrated into gem5 and can outperform the rule-based baseline in the evaluated canneal closed-loop case. The reward parameter produces different frequency-selection behaviors, suggesting a practical mechanism for distinguishing between more HighPerf-CNN and more conservative controllers. However, the present evaluation does not yet establish a cross-workload generalization, and the power-related conclusions should be interpreted as based on an approximate frequency-squared cost rather than a detailed hardware power model.

6.2 Limitations

Several limitations of the current work should be acknowledged.

The training samples are drawn from a full-system simulation of canneal. Ferret and blackscholes show little sensitivity to NoC frequency scaling, meaning that the training signal for distinguishing between frequency levels is concentrated in the canneal runs. The model's ability to generalize to workloads with qualitatively different communication patterns has not been validated.

The power analysis relies on the approximation rather than an accurate power model. This means that the energy efficiency comparison between strategies is indicative rather than quantitatively precise. Integration of a detailed model such as DSENT would be required before making hardware-level power claims.

Finally, the current implementation applies a single frequency decision to the entire NoC fabric, rather than controlling individual routers or sub-regions independently, exploiting spatial heterogeneity that a single global decision cannot capture

6.3 Future Work

Several directions present themselves as natural extensions of this work.

Expanding the evaluation to benchmarks with different levels of network sensitivity would allow the conditions under which adaptive DVFS provides meaningful benefit to be characterized more precisely.

Replacing the f^2 approximation with a power model, such as DSENT or Orion would enable end-to-end evaluation of energy and latency and give the λ parameter a concrete physical interpretation, allowing the controllers to be assessed on a single metric.

The spatial heatmap analysis revealed a pronounced center-heavy traffic distribution under XY routing, with central routers sustaining times the load of corner routers. Extending the controller to per-cluster or per-region frequency domains could allow high-load central routers to operate at higher frequency while peripheral routers reduce power.

References

[1] R. Bhargava and K. Troester, "Amd next-generation "zen 4" core and 4th gen amd epyc server cpus," *IEEE Micro*, vol. 44, no. 3, pp. 8–17, 2024. doi: 10.1109/MM.2024.3375070

[2] Intel Corporation, "Intel® Xeon® 6 Product Brief," Intel Product Documentation, 2024, accessed: May 2025. [Online]. Available: https://www.intel.com/content/www/us/en/products/docs/xeon-6-product-brief.html

[3] M. F. Reza, "Machine learning enabled solutions for design and optimization challenges in networks-on-chip based multi/many-core architectures," *J. Emerg. Technol. Comput. Syst.*, vol. 19, no. 3, Jun. 2023. doi: 10.1145/3591470. [Online]. Available: https://doi.org/10.1145/3591470

[4] Intel Corporation, "Intel® Xeon® Processor E5-2699 v4 Product Specifications," 2016, accessed: May 2025. [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/91317/intel-xeon-processor-e52699-v4-55m-cache-2-20-ghz/specifications.html

[5] ——, "Intel® Xeon® Platinum 8180 Product Specifications," 2017, accessed: May 2025. [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/120496/intel-xeon-platinum-8180-processor-38-5m-cache-2-50-ghz/specifications.html

[6] ——, "Intel® Xeon® Platinum 8380 Product Specifications," 2021, accessed: May 2025. [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/212287/intel-xeon-platinum-8380-processor-60m-cache-2-30-ghz/specifications.html

[7] ——, "Intel® Xeon® Platinum 8592+ Product Specifications," 2023, accessed: May 2025. [Online]. Available: https://www.intel.com/cont

ent/www/us/en/products/sku/237261/intel-xeon-platinum-8592-processor-320m-cache-1-90-ghz/specifications.html

[8] Advanced Micro Devices, Inc., "AMD EPYC™ 7001 Series Processors," 2017, accessed: May 2025. [Online]. Available: https://www.amd.com/en/products/processors/server/epyc/7001-series.html

[9] ——, "AMD EPYC™ 7002 Series Processors," 2019, accessed: May 2025. [Online]. Available: https://www.amd.com/en/products/processors/server/epyc/7002-series.html

[10] ——, "AMD EPYC™ 7003 Series Processors," 2021, accessed: May 2025. [Online]. Available: https://www.amd.com/en/products/processors/server/epyc/7003-series.html

[11] ——, "AMD EPYC™ 4th Gen 9004 Series Processors," 2022, accessed: May 2025. [Online]. Available: https://www.amd.com/en/products/processors/server/epyc/4th-generation-9004-and-8004-series.html

[12] ——, "AMD Launches 5th Gen AMD EPYC CPUs: Maintaining Leadership Performance and Features for the Modern Data Center," Oct. 2024, accessed: May 2025. [Online]. Available: https://www.amd.com/en/newsroom/press-releases/2024-10-10-amd-launches-5th-gen-amd-epyc-cpus-maintaining-le.html

[13] M. Ali, M. Welzl, and M. Zwicknagl, "Networks on chips: Scalable interconnects for future systems on chips," in *2008 4th European Conference on Circuits and Systems for Communications*, 2008. doi: 10.1109/ECCSC.2008.4611685 pp. 240–245.

[14] W. J. Dally and B. Towles, "Route packets, not wires: on-chip inteconnection networks," in *Proceedings of the 38th annual design automation conference*, 2001, pp. 684–689.

[15] L. Shang, L.-S. Peh, and N. K. Jha, "Dynamic voltage scaling with links for power optimization of interconnection networks," in *The Ninth International Symposium on High-Performance Computer Architecture, 2003. HPCA-9 2003. Proceedings.* IEEE, 2003, pp. 91–102.

[16] A. K. Mishra, R. Das, S. Eachempati, R. Iyer, N. Vijaykrishnan, and C. R. Das, "A case for dynamic frequency tuning in on-chip networks," ser. MICRO 42. New York, NY, USA: Association for

Computing Machinery, 2009. doi: 10.1145/1669112.1669151. ISBN 9781605587981 p. 292–303. [Online]. Available: https://doi.org/10.1145/1669112.1669151

[17] L. Benini and G. De Micheli, "Networks on chip: a new paradigm for systems on chip design," in *Proceedings 2002 Design, Automation and Test in Europe Conference and Exhibition*, 2002. doi: 10.1109/DATE.2002.998307 pp. 418–419.

[18] ——, "Networks on chips: A new soc paradigm," *computer*, vol. 35, no. 1, pp. 70–78, 2002.

[19] N. E. Jerger, T. Krishna, and L.-S. Peh, *On-chip networks*. Springer Nature, 2022.

[20] J. Lowe-Power, A. M. Ahmad, A. Akram, M. Alian, R. Amslinger, M. Andreozzi, A. Armejach, N. Asmussen, B. Beckmann, S. Bharadwaj *et al.*, "The gem5 simulator: Version 20.0+," *arXiv preprint arXiv:2007.03152*, 2020.

[21] H. Weerasena and P. Mishra, "Security of electrical, optical, and wireless on-chip interconnects: A survey," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, no. 2, Feb. 2024. doi: 10.1145/3631117. [Online]. Available: https://doi.org/10.1145/3631117

[22] N. Agarwal, T. Krishna, L.-S. Peh, and N. K. Jha, "Garnet: A detailed on-chip network model inside a full-system simulator," in *2009 IEEE international symposium on performance analysis of systems and software*. IEEE, 2009, pp. 33–42.

[23] C. Sun, C.-H. O. Chen, G. Kurian, L. Wei, J. Miller, A. Agarwal, L.-S. Peh, and V. Stojanovic, "Dsent - a tool connecting emerging photonics with electronics for opto-electronic networks-on-chip modeling," in *2012 IEEE/ACM Sixth International Symposium on Networks-on-Chip*, 2012. doi: 10.1109/NOCS.2012.31 pp. 201–210.

[24] C. Bienia, S. Kumar, J. P. Singh, and K. Li, "The PARSEC benchmark suite: characterization and architectural implications," in *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2008. doi: 10.1145/1454115.1454128 pp. 72–81.

[25] N. L. Binkert, R. G. Dreslinski, L. R. Hsu, K. T. Lim, A. G. Saidi, and S. K. Reinhardt, "The m5 simulator: Modeling networked systems," *Ieee micro*, vol. 26, no. 4, pp. 52–60, 2006.

[26] M. M. Martin, D. J. Sorin, B. M. Beckmann, M. R. Marty, M. Xu, A. R. Alameldeen, K. E. Moore, M. D. Hill, and D. A. Wood, "Multifacet's general execution-driven multiprocessor simulator (gems) toolset," *ACM SIGARCH Computer Architecture News*, vol. 33, no. 4, pp. 92–99, 2005.

[27] C. Bienia and K. Li, "Fidelity and scaling of the parsec benchmark inputs," in *IEEE International Symposium on Workload Characterization (IISWC'10)*, 2010. doi: 10.1109/IISWC.2010.5649519 pp. 1–10.

[28] Y. Yao and Z. Lu, "Pursuing extreme power efficiency with ppcc guided noc dvfs," *IEEE Transactions on Computers*, vol. 69, no. 3, pp. 410–426, 2020. doi: 10.1109/TC.2019.2949807

[29] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[30] S. Kiranyaz, O. Avci, O. Abdeljaber, T. Ince, M. Gabbouj, and D. J. Inman, "1d convolutional neural networks and applications: A survey," *Mechanical systems and signal processing*, vol. 151, p. 107398, 2021.

[31] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th international conference on machine learning (ICML-10)*, 2010, pp. 807–814.

[32] J. Bai, F. Lu, K. Zhang *et al.*, "ONNX: Open neural network exchange," https://github.com/onnx/onnx, 2019.

[33] PyTorch Contributors, "Pytorch c++ api," https://docs.pytorch.org/cppdocs/, accessed: June 2026.

[34] ——, "Installing c++ distributions of pytorch," https://docs.pytorch.org/cppdocs/installing.html, accessed: June 2026.

[35] Microsoft, "ONNX Runtime execution providers," https://onnxruntime.ai/docs/execution-providers/, accessed: June 2026.

[36] I. J. Ratul, Y. Zhou, and K. Yang, "Accelerating deep learning inference: A comparative analysis of modern acceleration frameworks," *Electronics*, vol. 14, no. 15, p. 2977, 2025.

[37] ONNX Runtime developers, "ONNX Runtime," https://onnxruntime.ai/, 2021.

[38] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze *et al.*, "{TVM}: An automated {End-to-End} optimizing compiler for deep learning," in *13th USENIX symposium on operating systems design and implementation (OSDI 18)*, 2018, pp. 578–594.

[39] U. Y. Ogras and R. Marculescu, "Prediction-based flow control for network-on-chip traffic," in *Proceedings of the 43rd annual design automation conference*, 2006, pp. 839–844.

[40] G. P. Nychis, C. Fallin, T. Moscibroda, O. Mutlu, and S. Seshan, "On-chip networks from a networking perspective: Congestion and scalability in many-core interconnects," *ACM SIGCOMM computer communication review*, vol. 42, no. 4, pp. 407–418, 2012.

[41] V. Soteriou and L.-S. Peh, "Exploring the design space of self-regulating power-aware on/off interconnection networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 18, no. 3, pp. 393–408, 2007.

[42] X. Chen, Z. Xu, H. Kim, P. Gratz, J. Hu, M. Kishinevsky, and U. Ogras, "In-network monitoring and control policy for dvfs of cmp networks-on-chip and last level caches," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 18, no. 4, pp. 1–21, 2013.

[43] R. Hesse and N. E. Jerger, "Improving dvfs in nocs with coherence prediction," in *Proceedings of the 9th International Symposium on Networks-on-Chip*, 2015, pp. 1–8.

[44] Y. Yao and Z. Lu, "Fuzzy flow regulation for network-on-chip based chip multiprocessors systems," in *2014 19th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2014. doi: 10.1109/ASP-DAC.2014.6742913 pp. 343–348.

[45] X. Zhang, D. Dong, C. Li, S. Wang, and L. Xiao, "A survey of machine learning for network-on-chips," *Journal of Parallel and Distributed Computing*, vol. 186, p. 104778, 2024. doi: https://doi.org/10.1016/j.jpdc.2023.104778. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S074373152300148X

[46] T.-R. Lin, D. Penney, M. Pedram, and L. Chen, "A deep reinforcement learning framework for architectural exploration: A routerless noc case study," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 99–110.

[47] J.-Y. Won, X. Chen, P. Gratz, J. Hu, and V. Soteriou, "Up by their bootstraps: Online learning in artificial neural networks for cmp uncore power management," in *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2014, pp. 308–319.

[48] Q. Fettes, M. Clark, R. Bunescu, A. Karanth, and A. Louri, "Dynamic voltage and frequency scaling in nocs with supervised and reinforcement learning techniques," *IEEE Transactions on Computers*, vol. 68, no. 3, pp. 375–389, 2019. doi: 10.1109/TC.2018.2875476

[49] H. Zheng and A. Louri, "An energy-efficient network-on-chip design using reinforcement learning," in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–6.

[50] M. Clark, Y. Chen, A. Karanth, B. Ma, and A. Louri, "Dozznoc: Reducing static and dynamic energy in nocs with low-latency voltage regulators using machine learning," in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2020, pp. 1–11.

[51] B. Wang, Z. Lu, and S. Chen, "Ann based admission control for on-chip networks," in *2019 56th ACM/IEEE Design Automation Conference (DAC)*, 2019, pp. 1–6.

[52] J. Yin, S. Sethumurugan, Y. Eckert, C. Patel, A. Smith, E. Morton, M. Oskin, N. E. Jerger, and G. H. Loh, "Experiences with ml-driven design: A noc case study," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 637–648.

Appendix A

Checkpoint Workflow Details

A.1 Checkpoints Workflow

To manage the substantial computational overhead of full-system simulation, a three-phase simulation strategy is employed. In the first phase, KVM-accelerated execution is utilized to fast-forward through the operating system boot and application initialization at near-native speeds. The primary goal is to establish a checkpoint near the boundary of the ROI. However, KVM execution is inherently non-deterministic, which precludes a guaranteed reproducible ROI entry point. This leads to a critical trade-off regarding checkpoint intervals: frequent intervals result in excessive overhead where the majority of execution time is consumed by checkpoint restoration, while sparse intervals lead to significant temporal gaps that complicate the transition to subsequent phases. Consequently, locating a usable checkpoint requires iterative trials and careful calibration to account for the variance in ROI entry ticks across different runs.

In the second phase, the simulation is restored from the KVM checkpoint using the AtomicSimpleCPU model. Because the Atomic model is deterministic, it ensures reproducibility and simplifies the process of locating the exact ROI boundary. By incorporating log-based instrumentation, the precise tick at which the ROI is entered can be recorded. In practice, establishing this precise ROI checkpoint for complex benchmarks such as canneal or ferret requires approximately eight to ten hours of server time. Once the exact entry tick is identified, a subsequent run is executed using the abs-max-tick and checkpoint-at-end commands to generate a definitive, reproducible checkpoint at the ROI boundary, a process that typically requires a full day to complete.

In the third phase, the simulation is restored from this reproducible checkpoint under the TimingSimpleCPU model, which accurately models memory timing and allows for the collection of NoC traffic data. This model provides high-fidelity results and serves as a baseline for evaluating different clock frequencies and memory-level parallelism models. However, data collection latency scales significantly as the simulation progresses. For instance, by the time the simulation reaches the 400th data point, the time required to collect a single point can increase to approximately 1.5 hours. To mitigate this, the relative stage proportions derived from the timing runs are used as a reference for more efficient sampling using Atomic models where appropriate.

Attempts to further accelerate data collection by creating intermediate checkpoints during the TimingSimpleCPU execution often result in system instability. A primary cause of failure is the internal management of CPU objects within the gem5 hierarchy. Once a switch-cpu command is triggered, the new CPU objects are renamed within the system tree. When a simulation is restarted, the default scripts often fail to locate the execution context within the checkpoint file, leading to immediate crashes. Furthermore, the instability is rooted in the complexity of the in-flight microarchitectural state. During timing simulation, the system is populated with unresolved memory requests, active Miss Status Holding Registers, and queued data packets on the memory interconnects.

Capturing this highly active or ”dirty” state is technically challenging. Upon restoration, the simulator’s initialization mechanisms struggle to perfectly reconstruct these interrupted queues and bus states, which frequently leads to assertion faults or segmentation faults. Additionally, checkpoints taken under detailed timing models are extremely sensitive to the underlying hardware configuration. Restoration requires an exact match of all timing parameters, such as cache hierarchies and memory controller settings, used during the checkpoint creation. This lack of flexibility negates the primary utility of checkpointing for architectural exploration, as it prevents the use of a single checkpoint for comparative experiments involving varying timing parameters.

TRITA-EECS-EX-2026:389
Stockholm, Sweden 2026

www.kth.se